



Elliptic Curve Digital Signature Algorithm (ECDSA) On RISC-V based FPGA Platform

Nauman

**SCHOOL OF ELECTRICAL AND ELECTRONIC ENGINEERING
A DISSERTATION SUBMITTED IN PARTIAL FULFILMENT OF
THE REQUIREMENTS FOR THE DEGREE OF
MASTER OF SCIENCE IN INTEGRATED CIRCUIT DESIGN**

2018

Contents

Contents	i
Abstract	iii
Acknowledgements	iv
List of Figures	v
List of Tables	vi
1 INTRODUCTION	1
1.1 Motivation	1
1.2 Background	2
1.3 Objectives	5
1.4 Organization of Thesis	6
2 LITERATURE REVIEW	7
2.1 Elliptic Curve Cryptography	7
2.2 Elliptic Curve Digital Signature Algorithm	12
2.2.1 Key Pair Generation	12
2.2.2 ECDSA Signature Generation	13
2.2.3 ECDSA Signature Verification	14
2.3 RISC - V	16
2.4 RISC - V Instruction Set - Summary	17
2.4.1 Integer Base Computation Instructions	20
2.4.2 Multiplication and Division Extension	24
2.4.3 Atomic Extension	24
2.4.4 Compression Extension	25
2.5 RISC - V Custom Accelerator	26
2.5.1 RoCC Interface	27

2.6	Arty FPGA Evaluation Kit	30
3	RISC - V FPGA IMPLEMENTATION	31
3.1	Rocket Chip SoC	32
3.2	Freedom Peripherals	32
3.3	Xilinx IPs for Arty	36
3.4	Complete System	36
4	SOFTWARE IMPLEMENTATION PROFILING	41
4.1	Software Execution Time Results	41
4.2	Resource Utilization	43
5	HARDWARE ACCELERATOR	44
5.1	Adder 256 Bits	44
5.2	Subtractor 256 Bits	46
5.3	Barrel Shifter 256 Bits	47
5.4	Complete Accelerator	48
5.5	Accelerator New Instructions	49
5.5.1	doWrite	50
5.5.2	doRead	51
5.5.3	doCopy	51
5.5.4	doAdd	52
5.5.5	doSub	52
5.5.6	doSL	53
5.5.7	doSR	53
6	HARDWARE ACCELERATOR PROFILING	55
6.1	Accelerator Hardware Execution Time Results	55
6.2	Accelerator Resource Utilization	58
7	CONCLUSION AND RECOMMENDATION FOR FUTURE WORK	59
7.1	Conclusion	59
7.2	Recommendation for Future Work	61

Bibliography	62
---------------------	-----------

Abstract

Cryptography is the mathematical tool used to secure data communication against unauthorized access or manipulation. In the last two decades digital cryptography has become an essential component of most of the digital devices. Implementing cryptography tasks in digital environment costs time, money, and energy. The Elliptic Curve Digital Signature Algorithm (ECDSA) is the Digital Signature Algorithm (DSA) based on the elliptic curve, which uses a small key compared to the others public-key algorithms. ECDSA is the most suitable scheme for environments where processor power and storage are limited.

With the growing concern in digital security, implementation of DSA in embedded system is increasing. Although ECDSA is suitable for embedded system but its implementation is not simple and constrained by hardware resources. This is mainly due to unavailability of instructions that favour arithmetic of more than 32 bits or 64 bits in generally available micro controller that are usually the heart of these embedded systems.

In this thesis, we developed a low cost, power efficient micro controller based on open source RISC-V Instruction Set Architecture (ISA) processor that supports instruction suitable for the implementation of ECDSA. RISC-V ISA introduced in 2010 by University of California, Berkeley is currently rapidly gaining support from semiconductor industry big players and huge community of open source developers. Our RISC-V implementation is of 32 bits data width but support 256 bits Arithmetic operations due to the presence of Hardware Accelerator for the efficient implementation of ECDSA. The presence of Hardware Accelerator reduces the complexity of software routines that one has to face when implementing ECDSA on general purpose micro controller while providing better execution time.

Acknowledgements

Through this , I would like to thanks **ALMIGHTY ALLAH**, The most merciful , for providing me the skills and strength to accomplish this task, which would not have been achieved without His blessings.

I would like to thanks my mentor **Zhang Chunyan** for her sincere unconditional support, encouragement and guidance all the way up to the completion of this task. She has been my inspiration as I hurdle all the obstacles in the completion of this project.

I would also like to thanks **Wee Tue Fatt David** for reviewing and suggesting changes in this Thesis.

I am grateful to **Mr. Sie Boo Chiang**, Head of Development, Chip Card & Security , Infineon Technologies Asia Pacific Pte.Ltd Singapore for offering me this excellent opportunity to work on this project.

I would also like to express my gratitude to my University supervisor Associate Professor **Dr. Gwee Bah Hwee** whose encouragement, suggestions and personal guidance has been a great support throughout the project.

I am thankful to **SUPARCO, Pakistan** for sponsoring my Master studies and providing me with this great opportunity. I am grateful to General Manager **Mujahidullah Khan** for selecting me and trusting me with this opportunity.

I am also grateful to all my Professors, Teachers, Friends and Colleagues for their support.

Last but not the least, I am thankful to my family and my parents. There support and pray kept me determine and focus.

List of Figures

2.1	Elliptic Curve in Real Number Field with $a = -3$ and $b = 2.5$	8
2.2	Elliptic Curve in Finite Field $p = 15$ with $a = -3$ and $b = 2.5$	10
2.3	Key Pair Generation	13
2.4	Signature Generation	14
2.5	Signature Verification	15
2.6	RISC - V Instruction Formats	18
2.7	B-type and J-type	19
2.8	Types of Immediate Produced in RISC - V	19
2.9	Atomic Extension Instructions	25
2.10	Custom Extension Instructions	27
2.11	RoCC Interface	28
3.1	Rocket Chip SoC Block Diagram	33
3.2	Freedom Peripheral Block Diagram	35
3.3	System Block Diagram	40
5.1	Adder 256 Block Diagram	46
5.2	Adder & Subtractor 256 Block Diagram	47
5.3	Barrel Shifter 256 Block Diagram	48
5.4	Accelerator Block Diagram	49

List of Tables

2.1	Registers Purpose in Program Execution	18
2.2	Integer Register - Immediate Instructions	20
2.3	Integer Register - Immediate Instructions (U-type)	20
2.4	Integer Register - Register Instructions	21
2.5	Integer Conditional Branch Instructions	22
2.6	Integer Unconditional Jump Instructions	22
2.7	Load and Store Instructions	23
2.8	Standard Extension M Instructions	24
2.9	Core Control Signal	28
2.10	Register Mode Command Signals	29
2.11	Register Mode Response Signals	29
2.12	Memory Mode Request Signals	29
2.13	Memory Mode Response Signals	30
3.1	Pin Mapping Arty	36
3.1	Pin Mapping Arty	37
3.1	Pin Mapping Arty	38
3.1	Pin Mapping Arty	39
3.2	Peripheral Memory Addresses	39
3.3	Interrupt Map	39
4.1	ECDSA Time Profiling	42
4.2	Time Profiling of Subroutines	42
4.3	Resource Utilization	43
5.1	Accelerator Instruction Summary	54
6.1	Accelerator Instruction Timing Profile	56
6.2	WriteAcc & ReadAcc Execution Time	56
6.3	Time Profiling of Subroutines with Accelerator	57
6.4	Combine Routines Time Profiling	57
6.5	ECDSA Time Profiling with Accelerator	58

6.6 Resource Utilization Accelerator	58
--	----

Chapter 1

INTRODUCTION

1.1 Motivation

Cryptography is defined as study and application of techniques to make the communication secure and reliable between two parties in the presence of other unwanted parties. Modern Computer Technology has revolutionized old cryptography and gave birth to Digital cryptography. The rapid development of Information Technology and its application require information safety and encryption provided by the form of digital cryptography known as Digital Signature. Today Digital Signature are used in number of application including e-mail, electronic fund transfer, ATM and smart cards, electronic data interchange, data storage, software distribution and copy right protection. Ref [1] discusses about the role of digital cryptography in providing services such as Confidentiality, Integrity, Authentication, Non-Repudiation and Copy Right Protection. Digital Signatures are based on Standard Digital Signature Algorithm (DSA). Digital Signature Algorithm involves complex computation leading to high consumption of processor power and memory. Thus, it is not suitable for Embedded System Application running on general available Micro - controller.

Elliptic Curve Digital Signature Algorithm (ECDSA) is a variant of Digital Signature Algorithm that is suitable for environment where processor power and memory resources are limited due to its smaller key size and higher security. Another widely used DSA is RSA named after the first initials of its inventors Rivest, Shamir and Adleman. There are well established methods proposed in literature for ECDSA implementation based on Software Design running on general purpose processors [2], Hardware Design running on FPGA [3] and Hardware Software Co - designed [4]. However, when it comes to Embedded System running on Micro - controller all these applications either require complete software design which takes too much time and is difficult to implement or a dedicated ECDSA processor that is required to be interfaced with the main controller using some standard data transfer protocol such as Serial, I²C, SPI etc. This shortcoming is mainly due to the unavailability of micro - controller with dedicated instruction to help out in implementation of ECDSA. We will address this problem in this thesis.

1.2 Background

Elliptic Curve Digital Signature Algorithm (ECDSA) consists of three main operations: Key Pair Generation, Signature Generation and Signature Verification. These three operations are achieved using Scalar Multiplication, Hash Generation, Point Addition, Random Number Generation and Arithmetic Operators. In 2013, Nabil, Lamia and Lotfi [5] suggested that the security level of ECDSA can be enhanced by tuning elementary modules for Scalar point multiplication, Hash function and Random number generation. They presented an optimized implementation example based on key size 233 bits, Montgomery point multiplication and SHA - 224 Hash function. In 2014, Abidi, Bouallegue and Kahri [6] reported a highly efficient implementation of ECDSA suitable for the FPGA. They also showed that ECDSA provides a higher level of security

as compare to their counterpart RSA for the same level of key size. These design consists of six modules; 163 - bits processor, SHA - 256 hash generator, Random number generator, Intermediate register, Control Unit and Arithmetic operator (ALU). In 2015, Ref [4] reported a hardware and software co - design of ECDSA on Xillinx FPGA with MicroBlaze processor. Bhanu, Deval and Mehta in [4] shifted generation of key and part of computation require for scalar point multiplication from hardware to software to reduce the hardware resource utilization and to increase the throughput of over all system. In 2016, Sghaier, Zeghid, and Machhout [7] reported the fast hardware implementation of ECDSA on Xillinx Virtex 5 FPGA platfrom. There implementation consisted of Hash function generator that can generate two Hash Function SHA - 256 and SHA - 512. They also made use of Projective Coordinates for Point Multiplication to ease the computation. In 2017, Sghaier et al. extended their work in [8] to reduce power and area. They optimize point multiplication, point addition and point doubling to accelerate the computation. They showed that these implementation are suitable for applications with low bandwidth communication, low storage and low computation environment.

Implementation of ECDSA in Embedded Devices running on General Purpose Micro - controller is not simple due to limitation of resources. Ref [2] studied software implementations of ECDSA and examined how some architectural features, such as word size and ISA affect its performance. Shi and Yan in [2] concluded that the better implementation of ECDSA on these Micro - controller requires extension in ISA with specific instruction to support point addition and point multiplication. Ref [9] authors reported an ECDSA processor that can act as an autonomous subsystem in embedded devices. Glas et al. in [9] followed the modular and scheduler approach so that the same resource can be re-utilized if require. As an application, they reported successful integration of there ECDSA block into vehicle to - X (V2X) communication system. Ref [10] by Noura et al. suggested another implementation of ECDSA for embedded devices using 32 bits data path. They studied there implementation in both FPGA and

ASIC form to conclude that ASIC form consume less power and is more efficient as compare to FPGA.

In terms of architecture majority of commercially available Micro - controller are based on Reduced Instruction Set Computer (RISC) running on closed source Instruction Set Architecture (ISA) such as x86 and ARM. These ISA are maintained by their vendors such as Intel, IBM, AMD etc and require license if they are to be used in any system. In 2014, Asanović and Patterson from University of California at Berkeley presented the case for free ISA [11]. They argued that the ISA are kept as proprietary just because of business reasons and there is no good technical reason for the unavailability of open free ISA. They floated the idea of real free open market of processor designs for the rapid growth and benefit of overall industry.

RISC - V is an open source ISA maintain by RISC - V Foundation and supported by many developers. RISC - V started as academic project focusing on academic teaching and research but it has gain high industrial support in the recent years. Current member of RISC - V foundation includes industry giants such as Google, AMD, IBM, SAMSUNG, Qualcomm, MediaTek, NVIDIA, Microsemi, Huawei, Micron Technology, NXP, Bluespec, SiFive, Siemens, GLOBALFOUNDRIES and many other. Ref [4] reported that the open source RISC - V from Berkeley is comparable and in some cases superior to closed source ARM (Cortex-A5).

RISC - V foundation [12] provides official document for User level and Privileged level RISC - V instruction set specification. They also maintain software tool chains for Program Compilation, Simulation and Debugging. There are number of open source implementation of RISC - V cores and System on Chip (SoC) currently available. Rocket Chip [13] started by University of California at Berkeley, is an open-source System on Chip design generator that emits synthesizable Register Transfer Logic (RTL). Rocket Chip is highly paramaterizable and can be used to generate different variant of RISC - V including single core and multi-cores. Rocket-chip generator can be used to generate

a high - performance cycle - accurate Verilator, Verilog RTL optimized for FPGAs and Verilog RTL for VLSI. SiFive [14] provide implementation example of RISC - V on low cost FPGA's platform based on Rocket Chip. Rocket Chip has been taped out at least eleven times and has yielded functional silicon prototypes.

1.3 Objectives

The work of this thesis can be considered as an extension to [4]. The main difference lies in the design of processor use to run the software routines. First of all in this work instead of using MicroBlaze processor we design our own variant of RISC - V 32-bits processor using Rocket Chip Generator. Secondly, we will first do profiling of software implementation of ECDSA running on RISC - V processor to identifies the routine which consume time and can benefit from Hardware Accelerator. Finally, we design Hardware Accelerator to improve performance of ECDSA on RISC - V processor. Software implementation we use is the proprietary of Infineon Technologies. Our final design is a complete Micro - controller with ECDSA Hardware Accelerator and have other generally available peripheral such as General purpose input/output (GPIO), Universal asynchronous receiver - transmitter (UART), Serial Peripheral Interface Bus etc. Although the design is prototype on FPGA platform but it can be tape out if require.

1.4 Organization of Thesis

The remainder of thesis is organized as follows. In Chapter 2, we discuss all the technical terms and preliminary theories involve including Elliptic Curve Cryptography and Elliptic Curve Digital Signature Algorithm followed by introduction of RISC - V and Summary of RISC - V Instruction Set Architecture. We conclude Chapter 2 with details of interface provided by RISC - V ISA and Rocket Chip for the integration of Custom Accelerator and some main features of Arty FPGA Evaluation Kit. In Chapter 3, we put forward complete detail of RISC - V System on Chip (SoC) implemented on Arty. In Chapter 4, we present profiling results of Software implementation of ECDSA running on RISC - V. In Chapter 5, we give detail of Hardware Accelerator modules and implementation. In Chapter 6, we present profiling results of Software implementation of ECDSA running on RISC - V with Hardware Accelerator. We end this thesis with Chapter 7 in which we summarize and conclude our work.

Chapter 2

LITERATURE REVIEW

The title of the thesis involves three important terms that are ECDSA, RISC - V and FPGA. In this chapter, we summarize the background theories and work of other authors mention in bibliography as per our work requirements that will be given in Chapter 3 and 5. We start our discussion with Elliptic Curve Cryptography in Section 2.1 which is the basis for Elliptic Curve Digital Signature Algorithm presented in Section 2.2. We then introduce RISC - V in Section 2.3 followed by the summary of RISC - V Instruction Set in Section 2.4. We also discuss Instructions and Interface provided by RISC - V for the integration of Custom Accelerator in Section 2.5. At the end, we present a general introduction and properties of Arty FPGA Evaluation Kit used for prototyping in Section 2.6.

2.1 Elliptic Curve Cryptography

Elliptic Curve Cryptography (ECC) is an asymmetric cryptography presented by Neal Koblitz and Victor S. Miller in 1984, defined over an elliptic curve. Elliptic curve are define by their domain parameters such as size of the chosen field p , Real number

coefficient a and b , Base Point G and Order of base point n . There are different form of elliptic curve depending upon the domain parameters but the one we are going to talk about is known as Weierstrass normal form for elliptic curves and is given by the following equation in real number field:

$$y^2 = x^3 + ax + b \quad (2.1)$$

where a and b are curve constant such that $4a^3 + 27b^2 \neq 0$. Figure 2.1 shows an elliptic curve in real number field.

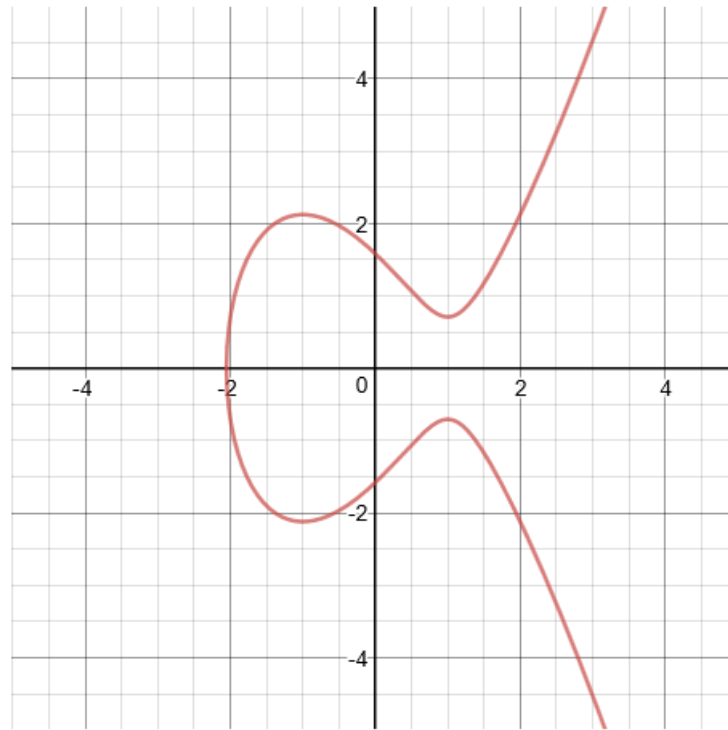


FIGURE 2.1: Elliptic Curve in Real Number Field with $a = -3$ and $b = 2.5$

All the points on elliptic curve are the member of Abelian Group. Therefore, they follow following properties:

- ♦ Any operation on a point on a curve gives another point that also lies on the curve.
- ♦ The identity element for a point is the point at infinity O i.e for any point P we have $P + O = P$.
- ♦ Every point except the one on x-axis has an inverse that is symmetric about x-axis i.e for any point define as $P = (x, y)$ we have $-P = (x, -y)$.
- ♦ Addition of three point P, Q and R is given as $P + Q + R = 0$.
- ♦ Point follows law of Associativity in Addition i.e $(P + Q) + R = P + (Q + R)$.
- ♦ Point follows law of Commutativity in Addition i.e $P + Q = Q + P$.

From the these properties we can deduce that if we multiply a point P on Elliptic Curve E with a scalar k then we get a point Q such that $Q = k * P$ and Q also lies on Elliptic curve E . This operation is known as **Scalar Multiplication** and form the basis for Elliptic Curve Cryptography. It has been shown mathematically that one can find Q if they know P and k . But it is hard to find k if Q and P are known. This is known as **Elliptic Curve Discrete Logarithm Problem** (ECDLP). The security provided by ECC lies in the difficulty of ECDLP. More difficult the ECDLP more secure is ECC.

Scalar Multiplication can be achieved using Geometric and Algebraic method and it consist of two Operations; Point Addition and Point Multiplication.

Point Addition of two point $P(x_1, y_1)$ and $Q(x_2, y_2)$ on Elliptic curve E to find resulting point $R(x_3, y_3)$ can be performed using following algebraic equations.

$$x_3 = \left(\frac{y_2 - y_1}{x_2 - x_1} \right)^2 - x_1 - x_2 \quad (2.2)$$

$$y_3 = \left(\frac{y_2 - y_1}{x_2 - x_1} \right)(x_1 - x_3) - y_1 \quad (2.3)$$

Point Doubling is an addition of a point $P(x_1, y_1)$ to itself on Elliptic curve E to find resulting point $R(x_3, y_3)$ such that $R = P + P = 2P$. It can be performed using following algebraic equations.

$$x_3 = \left(\frac{3x_1^2 + a}{2y_1} \right)^2 - 2x_1 \quad (2.4)$$

$$y_3 = \left(\frac{3x_1^2 + a}{2y_1} \right)(x_1 - x_3) - y_1 \quad (2.5)$$

Elliptic Curve define over real number contain infinite number of elements and therefore it is impossible to implement ECC for practical digital applications. For practical application elliptic curve use are defined over finite field p . Figure 2.2 shows curve of Figure 2.1 in finite field of 15 elements.

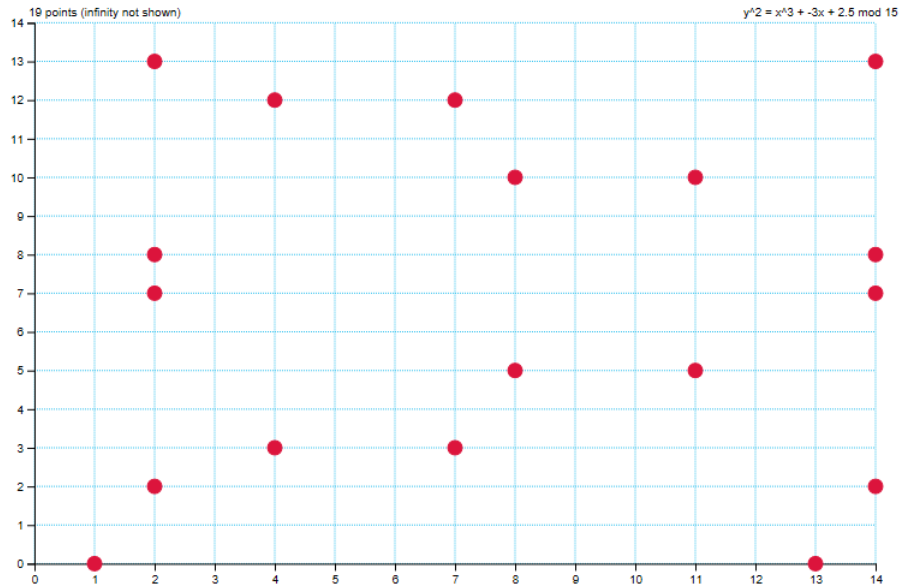


FIGURE 2.2: Elliptic Curve in Finite Field $p = 15$ with $a = -3$ and $b = 2.5$

There are two types of finite field mostly use in ECC; Binary Field F_{2^n} of characteristic two and Prime field F_p where p is usually a large prime number. With finite field Eq 2.1 is given as

$$y^2 = x^3 + ax + b \pmod{p} \quad (2.6)$$

where *mod* stands for arithmetic in modular form which basically means that no real number less than 0 and greater than p is allowed. So if in any operation result is negative it is adjusted by adding p in it. And if the result is greater than p it is adjusted by subtracting p from it until it becomes less than p .

Equations 2.2 - 2.5 for point addition and point doubling involve number of modular arithmetic addition, subtraction, multiplication and division. Usually division is computationally more intensive than other operations and require too much resources in actual implementation. Therefore, Affine coordinate point (x,y) are usually transformed into Projective coordinate point (X,Y,Z) . Various Types of Projective coordinates exist but two widely used are Standard projective coordinate and Jacobian projective coordinate.

Any point given in Affine coordinate $P(x,y)$ can be represented in Standard projective coordinate point $P'(X,Y,Z)$ by using following substitution:

$$x = \frac{X}{Z} \quad y = \frac{Y}{Z^2} \quad (2.7)$$

Any point given in Affine coordinate $P(x,y)$ can be represented in Jacobian projective coordinate point $P'(X,Y,Z)$ by using following substitution:

$$x = \frac{X}{Z^2} \quad y = \frac{Y}{Z^3} \quad (2.8)$$

Use of these transformations help to reduce the number of division operation required for point addition and point doubling. Once the computation is finished the resulting point in Affine coordinate can be obtained by reversing the substitution for X,Y and Z . Usually the initial value for $Z = 1$ to ease the computation. Reader can refer to

[15] for Equations 2.1 - 2.5 in Standard Projective Coordinate and Jacobian Projective Coordinate forms.

National Institute of Standards and Technology (NIST,USA) provides standard curve parameters in [16]. In our implementation we will use Curve $P - 256$.

2.2 Elliptic Curve Digital Signature Algorithm

Elliptic Curve Digital Signature Algorithm (ECDSA) is a variation of Digital Signature Algorithm based on Elliptic Curve Cryptography. It is accepted as Standard by International Standard Organization as ISO 14888-3, American National Standards Institute as ANSI X9.62 and Institute of Electrical and Electronics Engineers as IEEE 1363-2000. Signature generation and verification of message using ECDSA involves three different algorithm named as Key Pair Generation, ECDSA Signature Generation and ECDSA Signature Verification. Below we will discuss steps involve in each algorithm in detail.

2.2.1 Key Pair Generation

Key Pair Generation is responsible for the generation of pair of Public Key and Private Key. Public Key can be made available to the other but Private key remain specific to the user. Encryption done using Public Key can only be decrypted by Private Key. Generation of Key Pair requires a Random Number Generator and Modular Arithmetic Processor to perform Scalar Multiplication as shown in Figure 2.3.

In Figure 2.3, E is an Elliptic Curve define over prime field p with a and b as constant coefficient of curve. G is a base point and n is the order of the base point. Order n is define as a number that can be used for the scalar multiplication of base point G

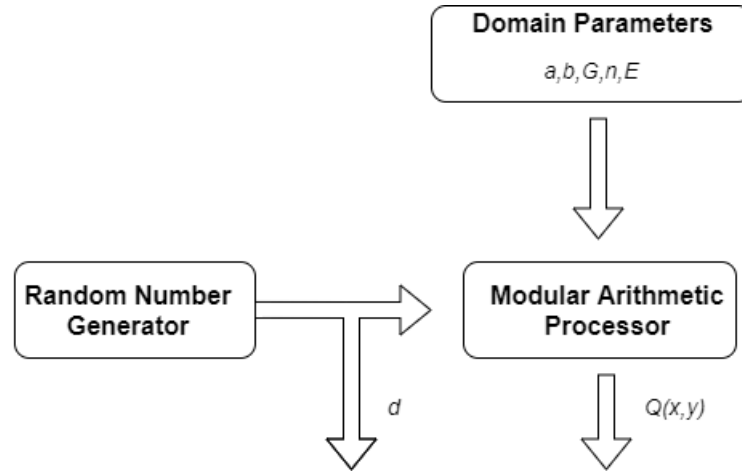


FIGURE 2.3: Key Pair Generation

before the point start repeating themselves due to modular arithmetic. Private key d is generated by Random number generator. Domain parameters (a, b, n, G, E) and Private key d are used by Modular Arithmetic processor to generate Public Key point Q that lies on curve E . Public Key Q is generated by scalar multiplication of private key d and base point G i.e. $Q = d * G$ as discuss in Section 2.1.

2.2.2 ECDSA Signature Generation

ECDSA Signature Generation is responsible for the generation of set of two number known as Signature specific to the input message. Signature generation make use of Private Key d generated by Key Pair generation to perform it operation. ECDSA Signature Generation requires Random Number Generator, Hash Function Generator and Modular Arithmetic processor as shown in Figure 2.4.

The random generator generates a number k . This number is then used for a scalar Multiplication of Base point G to obtain another point $R(x, y)$. The first member r of signature set is then calculated as $r = x \bmod n$ where n is the order of base point. If the resultant $r = 0$ then the whole procedure is repeated with some other value of k generated through random number generator. The message m is then passed through

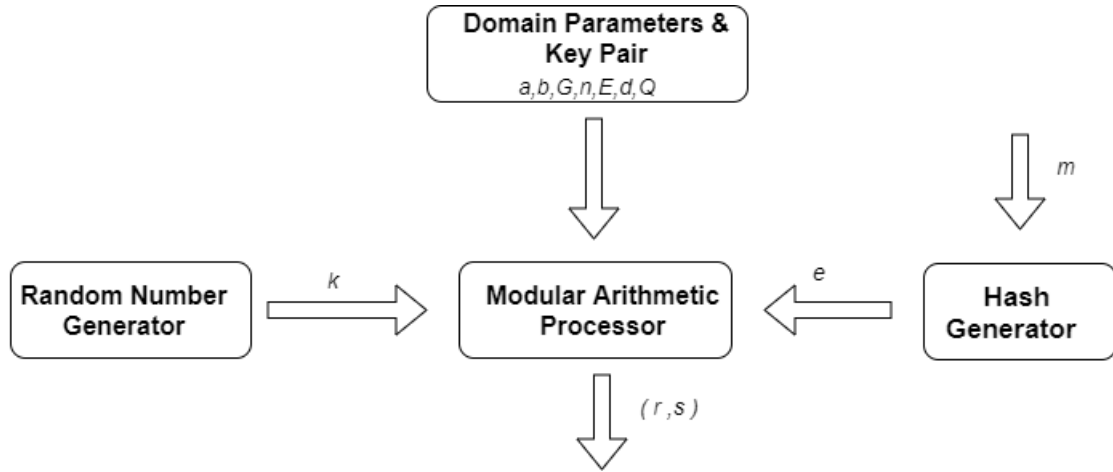


FIGURE 2.4: Signature Generation

hash function generator to get message digest $e = h(m)$ where $h()$ is a cryptographic hash generator. A cryptographic hash function is a One - way function that convert data of arbitrary size to fix size. Some of the commonly used standard hash function are SHA - 1, SHA - 2 and SHA - 3. These function differ on the amount of security they provide. The second member s of signature set is then obtain using following formula.

$$s = k^{-1}(e + d * r) \quad \text{mode } n \quad (2.9)$$

If $s = 0$ the whole procedure is repeated again with new value of k from random number generator.

2.2.3 ECDSA Signature Verification

ECDSA Signature Verification is responsible to verify that the message received with signature set is not fake. It makes use of Private Key to do so. Since it generates local copy of signature pair therefore the computation involved in Signature Verification

are more complex and almost double as compare to Signature generation. But like Signature generation this step also uses Random Number Generator, Hash Function Generator and Modular Arithmetic processor as shown in Figure 2.5

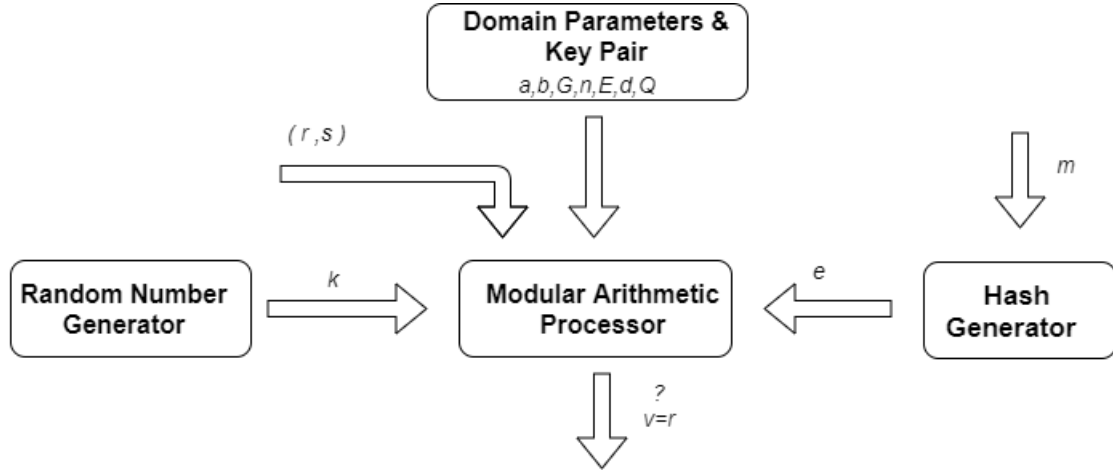


FIGURE 2.5: Signature Verification

The message m receive with signature set (r, s) is passed through the hash generator to get message digest $e = h(m)$. This message digest is then used to calculate two scalar value u_1 and u_2 such that

$$u_1 = e * s^{-1} \mod n \quad (2.10)$$

$$u_2 = r * s^{-1} \mod n \quad (2.11)$$

u_1 and u_2 are used to find point $R(x, y)$ such that

$$R = u_1 * G + u_2 * Q \quad (2.12)$$

where G is the base point and Q is the public key. Now the require local copy v of signature is generated using $v = x \mod n$. The signature is correct if the generated local copy v is the same as r .

2.3 RISC - V

There are two architectures approach usually followed in the design of a processor; CISC and RISC. CISC stands for Complex Instruction Set Computers. CISC emphasis on hardware resources. It has instructions that can perform complex task and requires multiple clock. Therefore the hardware implementation of CISC is simple but consume much space and resources as compare to RISC. On the other hand, RISC stands for Reduced Instruction Set Computer. RISC emphasis more on software design and provide only basic single cycle instruction that can be combine to achieve complex task. Therefore, actual hardware implementation of RISC is complex but consume much less space and resources as compare to CISC.

RISC - V is an open source implementation of RISC officially managed by RISC - V Foundation. RISC - V is not the first open source RISC implementation. There are other such as OpenRISC, OpenSPARC etc. But what make RISC - V different from the other is that it is equally applicable to high and low end computing processor, mobile phones and other embedded devices. The RISC - V ISA has been designed with low power real world implementations in mind without over - architecting for a particular device. One another main reason for the RISC - V success is it growing community of developers and industrial supporter.

The RISC - V Foundation is a recognised nonprofitmaking association to regulate, guard and encourage the open and free RISC - V instruction set architecture. RISC - V promotes use of RISC - V hardware and software ecosystem in all computing devices irrespective of their size and purpose. RISC - V Foundation hosts software tool chains and specification documents for ISA and provides hardware core and System on Chip example implementation of RISC - V. RISC - V Foundation Members participate and access in the development of the RISC - V ISA specifications, related hardware and software ecosystem. Complete RISC - V specification is lengthy and out of scope here.

Reader can refer to [17] and [18] for the complete documentation of RISC - V User level specification and Privileged level specification respectively. We will discuss some specifications of RISC - V ISA important to understand this thesis work in Section 2.4.

The RISC - V ISA is defined in the form of Integer (I) ISA which is essential for any implementation and is known as Base ISA. Base can be extended with standard optional extensions. Depending upon the width of register and size of address space there are three frozen base ISA; RV32-I, RV64-I and RV128-I for 32-bits, 64-bits and 128-bits register respectively. The base ISA is carefully restricted to a minimum set of instructions suitable to provide a realistic target for assemblers, linkers, compilers and operating systems. It consists of integer computational instructions, integer loads and stores, control flow instructions, memory model, system control and status register instructions. Base ISA can be extended with standard extension such as Integer Multiplication and Division (M), Atomic Instruction (A), Single - precision Floating - point (F), Double - precision Floating - point (D), Quad - Precision Floating - Point (Q), Decimal Floating - Point (L), Compressed Instructions (C), Bit Manipulation (B), Dynamically Translated Languages (L), Transactional Memory (T), Packed - SIMD Instructions (P), Vector Operations (V) and User-Level Interrupts (N). Not all standard extensions are frozen and are likely to be change in the future. Since most of the implementation make use of I,M,A,F,D therefore design with IMAFD is abbreviated as G where G stands for General-purpose scalar instruction set. There is one unfrozen base level ISA RV-32E intended for embedded systems under development.

2.4 RISC - V Instruction Set - Summary

Our hardware implementation make use of 32-bits integer register base RV32-I with Multiplication and Division (M), Atomic Instruction (A) and Compressed Instructions (C) standard extensions. RV32-I base has 32 (x0 - x31) 32-bits registers and 1 32-bits

Program counter (pc) register. Out of 32, 31 (x1 - x31) are general purpose registers while x0 register is hard-wired to zero. These registers perform specific operations in standard program execution as shown in Table 2.1.

TABLE 2.1: Registers Purpose in Program Execution

Register	Description	ABI Name
x0	Hard-wired zero	zero
x1	Return address	ra
x2	Stack pointer	sp
x3	Global pointer	gp
x4	Thread pointer	tp
x5	Temporary/alternate link register	t0
x6,x7	Temporaries	t1,t2
x8	Saved register/frame pointer	s0/fp
x9	Saved register	s1
x10-x11	Function arguments/return values	a0-a1
x12-x17	Function arguments	a2-a7
x18-x27	Saved registers	s2-s11
x28-x31	Temporaries	t3-t6

The base **Integer** ISA support four naturally aligned formats of instruction or addressing mode; Register to Register (R), Immediate to Register (I), Signed Immediate to Register (S) and Unsigned Immediate to Register (U). The RISC - V ISA keeps the source register rs1 and rs2 and destination register rd at the same position in all formats to simplify decoding except in S-type as shown in Figure 2.6. Depending upon

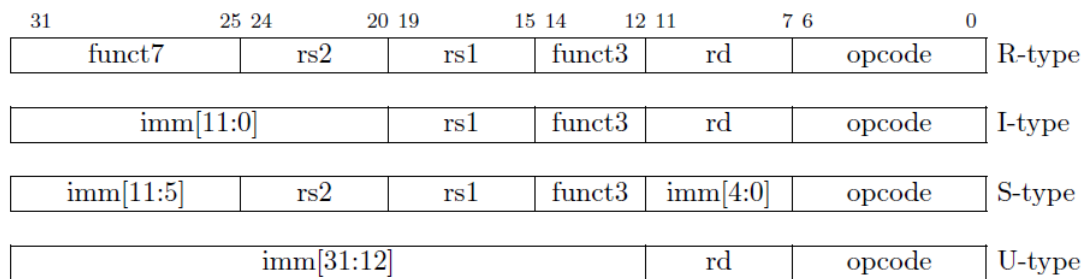


FIGURE 2.6: RISC - V Instruction Formats

the handling of immediate (Imm) there are two more variant of instruction format; Branch (B) and Jump (J) as shown in Figure 2.7.

imm[12]	imm[10:5]	rs2	rs1	funct3	imm[4:1]	imm[11]	opcode	B-type
imm[20]	imm[10:1]	imm[11]	imm[19:12]	rd	opcode	J-type		

FIGURE 2.7: B-type and J-type

The difference between the S-type and B-type formats is that the 12-bit immediate field in S-type used to encode branch offsets is divided into multiples of 2 in the B-type. The difference between the U-type and J-type formats is that the 20-bit immediate is shifted left by 12 bits to form U immediate and by 1 bit to form J immediate.

RISC - V has an asymmetric immediate encoding which means that the immediate are formed by concatenating different bits in an asymmetric order based on the specific immediate formats. Thus all instruction with immediate form an immediate version as shown in Figure 2.8.

31	30	20	19	12	11	10	5	4	1	0	
— inst[31] —						inst[30:25]	inst[24:21]	inst[20]	I-immediate		
— inst[31] —						inst[30:25]	inst[11:8]	inst[7]	S-immediate		
— inst[31] —					inst[7]	inst[30:25]	inst[11:8]	0	B-immediate		
inst[31]	inst[30:20]		inst[19:12]		— 0 —						U-immediate
— inst[31] —			inst[19:12]		inst[20]	inst[30:25]	inst[24:21]	0	J-immediate		

FIGURE 2.8: Types of Immediate Produced in RISC - V

2.4.1 Integer Base Computation Instructions

Integer base computation instructions are encoded as Register - immediate and Register - register. Register - immediate make use of I-type format and Register - register make use of R-type format. Table 2.2 gives the list of Instructions with I-type format with intended operation.

TABLE 2.2: Integer Register - Immediate Instructions

Instruction	Description	Operation
ADDI	Add Immediate	$rd = rs + Imm$
SLTI	Set Less Than Immediate	set $rd = 1$ if $rs < Imm$ otherwise $rd = 0$
SLTIU	Set Less Than Immediate Unsigned	Same as SLTI with Imm as unsigned
ANDI	Bitwise And Immediate	$rd = rs \& Imm$
ORI	Bitwise OR Immediate	$rd = rs Imm$
XORI	Bitwise XOR Immediate	$rd = rs \oplus Imm$
SLLI	Logical left shift Immediate	$rd = rs \ll Imm$
SRLI	Logical right shift Immediate	$rd = rs \gg Imm$
SRAI	Arithmetic right shift Immediate	$rd = rs \ggg Imm$

There are two particular immediate instruction LUI (Load Upper Immediate) and AUPIC (Add Upper Immediate to Program Counter) that use U-type format instead of I-type. These instructions are explained in Table 2.3.

TABLE 2.3: Integer Register - Immediate Instructions (U-type)

Instruction	Description	Operation
LUI	load upper immediate	$rd = Imm$
AUPIC	add upper immediate to program counter	$pc = pc + Imm$

RV32-I provide several arithmetic instructions based on R-type format. These instructions read value from source register $rs1$ and $rs2$ and provide result in destination register rd as shown in Table 2.4.

TABLE 2.4: Integer Register - Register Instructions

Instruction	Description	Operation
ADD	Add registers	$rd = rs1 + rs2$
SUB	Subtract registers	$rd = rs1 - rs2$
SLT	Set Less Than register	set $rd = 1$ if $rs1 < rs2$ otherwise $rd = 0$
SLTU	Set Less Than register Unsigned	same as SLT with registers as unsigned
AND	Bitwise And register	$rd = rs1 \& rs2$
OR	Bitwise OR register	$rd = rs1 rs2$
XOR	Bitwise XOR register	$rd = rs1 \oplus rs2$
SLL	Logical left shift register	$rd = rs1 \ll rs2$
SRL	Logical right shift register	$rd = rs1 \gg rs2$
SRA	Arithmetic right shift register	$rd = rs1 \ggg rs2$

RV32-I base provides two types of control flow instructions; Conditional branches and Unconditional jumps. Conditional branch jump is taken when some conditions are met as a result of some logical operation performed between register to register or register to immediate. Table 2.5 list conditional branch instructions supported by RV32-I. All conditional branch instructions make use of B-type format. Conditional branch jump can range between $\pm 4\text{KiB}$.

RV32-I base support two unconditional branch instruction JAL (Jump and link) and JALR (Jump and link register). JAL is based on J-type format. JALR is based on I-type format. Table 2.6 gives information about both instructions. These instruction can jump about $\pm 1\text{MiB}$.

RISC - V is a load - store architecture which means that only load and store instructions can access memory and transfer value between memory and register. This is done to increase the speed of general operations of processor since the memory access is

TABLE 2.5: Integer Conditional Branch Instructions

Instruction	Description	Operation
BEQ	Branch if Equal	$pc = pc + Imm$ if $rs1 = rs2$
BNQ	Branch if not Equal	$pc = pc + Imm$ if $rs1 \neq rs2$
BGE	Branch if Greater	$pc = pc + Imm$ if $rs1 \geq rs2$
BGEU	Branch if Greater Unsigned	$pc = pc + Imm$ if $ rs1 \geq rs2 $
BLT	Branch if Lesser	$pc = pc + Imm$ if $rs1 \leq rs2$
BLTU	Branch if Greater Unsigned	$pc = pc + Imm$ if $ rs1 \leq rs2 $

TABLE 2.6: Integer Unconditional Jump Instructions

Instruction	Description	Operation
JAL	Jump and link	$pc = pc + Imm$
JALR	Jump and link register	$pc = pc + rs1 + Imm$

slow. RV32-I provides 32-bits address space that is little endian and byte addressable. Loads instructions make use of I-type format and Store instructions make use of S-type format. Table 2.7 list Load and Store instruction supported by RV32-I base.

Apart from arithmetic instruction and control flow instructions base RV32-I also provide System Control and Status register instructions (CSR). These instruction require privileged level access and are designed using I-type format. The details of CSR can be found in [17] and [18]. For our work we make use of RDCYCLE and RDINSTRET for software profiling. RV32-I provides a number of 64-bit read - only user - level counters, which are mapped into the 12-bit CSR address space and accessed in 32-bit pieces using CSRRS (CSR read in Register) instructions. RDCYCLE holds a count of the number of clock cycles executed by the processor core during the current thread

TABLE 2.7: Load and Store Instructions

Instruction	Description	Operation
LW	Load Word	Load 32 bit value from memory to rd
LH	Load Half Word	Load 16 bit value from memory and sign extend to 32 bit before storing it into rd
LHU	Load Half Word Unsigned	Load 16 bit value from memory and zero extend to 32 bit before storing it into rd
LB	Load Byte	Load 8 bit value from memory and sign extend to 32 bit before storing it into rd
LBU	Load Byte Unsigned	Load 8 bit value from memory and zero extend to 32 bit before storing it into rd
SW	Store Word	Store 32 bit value from rs2 to memory location index by rs1
SH	Store Half Word	Store lower 16 bit value from rs2 to memory location index by rs1
SB	Store Byte	Store lower 8 bit value from rs2 to memory location index by rs1

starting from an arbitrary start time in the past. RDINSTERT counts number of instructions retired by current thread in the processor starting from some arbitrary start point in the past.

RISC - V base support multiple hardware thread also known as Hart. Memory synchronization between different thread is made possible by FENCE and FENCE.I instructions. RISC - V has a relaxed memory model between harts, requiring a FENCE instruction to guarantee ordering between memory operations from different RISC - V harts. FENCE instruction basically ensure that memory accesses initiated before the fence instruction are completed and their result is visible to the processor before continuing. The FENCE.I instruction is used to synchronize the instruction and data streams. With FENCE.I RISC - V guarantee that stores to instruction memory will be made visible to instruction fetches on the same hart.

2.4.2 Multiplication and Division Extension

Multiplication and Division (M) is a standard extension that provides integer Multiplication, Division and Remainder operations on the integer stored in register. All RV32-M instructions are R-Type. Table 2.8 provides introduction of M instructions available for RV32.

TABLE 2.8: Standard Extension M Instructions

Instruction	Description	Operation
MUL	Multiply registers low	Multiply rs1 and rs2 resultant is 64-bits, store lower 32 bits in rd
MULH	Multiply registers high	Multiply rs1 and rs2 resultant is 64-bits, store higher 32 bits in rd
MULHU	Multiply registers high unsigned	Multiply unsigned rs1 and rs2 resultant is 64-bits, store higher 32 bits in rd
MULHSU	Multiply registers high signed unsigned	Multiply signed rs1 and unsigned rs2 resultant is 64-bits, store higher 32 bits in rd
DIV	Divide register	$rd = rs1 \div rs2$
DIVU	Divide register unsigned	$rd = rs1 \div rs2 $
REM	Find Remainder	$rd = rs1 \% rs2$
REMU	Find Remainder unsigned	$rd = rs1 \% rs2 $

2.4.3 Atomic Extension

Atomic instructions (A) are special hardware instructions that perform an operation on one or more memory locations atomically. These instructions help to synchronize read - modify - write memory between different harts running in same memory space. The two forms of atomic instruction provided by this extension are load - reserved or store - conditional instructions and atomic fetch - and - op memory instructions. Both types of instruction support various memory consistency orderings including unordered, acquire, release, and sequentially consistent semantics. Load - reserved (LR) and store -

conditional (SC) instructions perform atomic memory operations on a single memory word. LR loads a word from the address in rs1, places the sign - extended value in rd, and registers a reservation on the memory address. SC writes a word in rs2 to the address in rs1, provided a valid reservation still exists on that address. SC writes zero to rd on success or a non - zero code on failure. The atomic memory operation instructions perform read - modify - write operations for multiprocessor synchronization and are encoded with an R - type instruction format. These instructions atomically load a data value from the address in rs1, place the value into register rd, apply a binary operator to the loaded value and the original value in rs2, then store the result back to the address in rs1. Figure 2.9 shows these two types of instructions.

00010	aq	rl	00000	rs1	010	rd	0101111	LR.W
00011	aq	rl	rs2	rs1	010	rd	0101111	SC.W
00001	aq	rl	rs2	rs1	010	rd	0101111	AMOSWAP.W
00000	aq	rl	rs2	rs1	010	rd	0101111	AMOADD.W
00100	aq	rl	rs2	rs1	010	rd	0101111	AMOXOR.W
01100	aq	rl	rs2	rs1	010	rd	0101111	AMOAND.W
01000	aq	rl	rs2	rs1	010	rd	0101111	AMOOR.W
10000	aq	rl	rs2	rs1	010	rd	0101111	AMOMIN.W
10100	aq	rl	rs2	rs1	010	rd	0101111	AMOMAX.W
11000	aq	rl	rs2	rs1	010	rd	0101111	AMOMINU.W
11100	aq	rl	rs2	rs1	010	rd	0101111	AMOMAXU.W

FIGURE 2.9: Atomic Extension Instructions

2.4.4 Compression Extension

Compression extension (C) reduces static and dynamic code size by adding short 16-bits instruction encodings for common operations. It uses a simple compression scheme that offers shorter 16-bits versions of common 32-bits RISC - V instructions when:

- ♦ The immediate or address offset is small.

- ♦ One of the used registers is x0, x1, or x2.
- ♦ The destination register rd and the first source register rs1 are identical.
- ♦ The registers used are the 8 most popular ones (x8 - x15).

The C extension is compatible with all other standard instruction extensions but can not be used stand alone. All compressed instructions are denoted by *C*. in the start e.g C.LW is compressed version of 32-bits compressed load word (LW) instruction.

RISC - V ISA supports three thread mode; User/Application (U), Supervisor (S) and Machine (M). Depending upon the mode hart is running, changes access to the System Control and Status register. Machine level mode has the highest priority and is the only mandatory mode for a RISC - V hardware platform. It is used for the low level access to a hardware platform. This is also the default mode on reset. Our hardware implementation also runs in M - mode.

2.5 RISC - V Custom Accelerator

RISC - V ISA is designed to ease the implementation of Custom Accelerator and Co-processor. Basic ISA support 4 opcode's for Custom Accelerator implementation. Major opcode's marked as Custom - 0 and Custom - 1 are frozen and recommended for use by custom instruction set extensions within the base 32-bits instruction format. The opcode's marked Custom - 2 and Custom - 3 are reserved for future use by RV128-I, but can be used for custom instruction-set extensions in RV32-I and RV64-I. Figure 2.10 provide Register - register instruction set format for custom accelerators.

These custom instructions ensure that as long as we are building accelerator based on these instruction no changes in software tool chain is required. This makes the software integration for Custom Accelerator easy. For Hardware integration we have to

31	27	26	25	24	20	19	15	14	12	11	7	6	0	
funct7				rs2		rs1		funct3		rd		0001011		Custom - 0
funct7				rs2		rs1		funct3		rd		0101011		Custom - 1
funct7				rs2		rs1		funct3		rd		1011011		Custom - 2
funct7				rs2		rs1		funct3		rd		1111011		Custom - 3

FIGURE 2.10: Custom Extension Instructions

see how the accelerator is going to interface with main processor, registers and memory. Luckily Rocket Chip SoC implementation provide Rocket Custom Coprocessor Interface (RoCC) that can be used for the hardware integration of Custom Accelerator.

2.5.1 RoCC Interface

RoCC interface provides signal used to implement fully functional RISC - V accelerator. RoCC interface is divided into two categories; Default and Extended. Default RoCC Interface represents signals that are basic and necessary such as Core control, Register mode and Memory mode. Extended RoCC Interface represents signals used by the advance application such as Uncached Tile Link for communication between an accelerator and L2 memory, Floating Point Unit (FPU) for an accelerator to send and receive data from an FPU, Control Status Register used by Linux on the core to recognize the accelerator and Page Table Walker for address translation from an accelerator. Figure 2.11 shows currently available RoCC interface. [19] provide detail information about the RoCC interface signals.

Core Control (CC) is the default RoCC interface that ensure coordination between main processor core and accelerator. Signals available in Core control interface are shown in Table 2.9 with description. Register mode (Core) is the default RoCC interface that perform exchange of data between an accelerator and core. It consist of two subclass signals RoCC Command and RoCC Response as shown in Table 2.10 and Table 2.11 respectively.

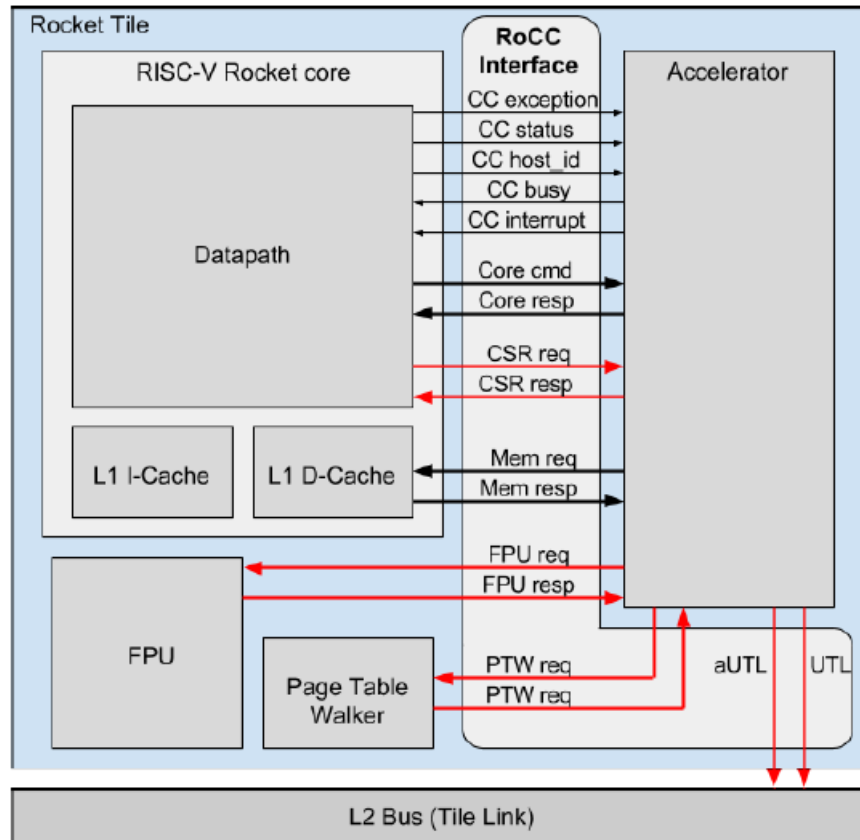


FIGURE 2.11: RoCC Interface

TABLE 2.9: Core Control Signal

Direction	Signal name	Description
output	busy	Set during in-flight memory requests
input	status	Set when a privileged process runs on the core
output	interrupt	Set for invalid instructions or to indicate any problem
input	exception	Set by core to trigger exception behaviour
input	host-id	Used to distinguish between command packets from different hosts.

Memory mode (Mem) is the default RoCC interface that perform communication between an accelerator and L1-D Cache. It is composed of Memory request and Memory response subgroups as shown in Table 2.12 and Table 2.13 respectively.

TABLE 2.10: Register Mode Command Signals

Direction	Signal name	Description
output	ready	Control lines
input	valid	Control lines
input [6:0]	function	For different kinds of accelerator instructions.
input [4:0]	rs2	Source register rs2 IDs
input [4:0]	rs1	Source register rs1 IDs
input	xd	Set if destination register exists
input	xs1	Set if source register rs1 exist
input	xs2	Set if source register rs2 exist
input [4:0]	rd	Destination register ID
input [6:0]	opcode	Custom instruction opcode used in case of multiple accelerators.
input [63:0]	rs1-data	Source register rs1 data
input [63:0]	rs2-data	Source register rs2 data

TABLE 2.11: Register Mode Response Signals

Direction	Signal name	Description
input	ready	Control lines
output	valid	Control lines
output[4:0]	rd	Destination register ID in the response
output[63:0]	data	Destination register data in the response

TABLE 2.12: Memory Mode Request Signals

Direction	Signal name	Description
input	ready	Control lines
output	valid	Control lines
output[39:0]	addr	Memory address corresponds to read/write
output[9:0]	tag	Unique identity assigned to every memory request
output[4:0]	cmd	Memory request opcode [0x0000=load,0x0001=store]
output[2:0]	typ	Width of response [0x000=8-bits,0x001=16-bits, 0x010=32-bits,0x011=64-bits].
output	phys	De-asserted if addresses are virtual and need translation
output[63:0]	data	Store data

TABLE 2.13: Memory Mode Response Signals

Direction	Signal name	Description
input	ready	Control lines
input [39:0]	addr	Displays load/store request address
input [9:0]	tag	To differentiate between responses to multiple requests.
input [4:0]	cmd	Returns command code of request
input [2:0]	typ	Indicates width of the data in response
input [63:0]	data	Contains data response to a load request
input	nack	(To be known)
input	replay	(To be known)
input	has-data	Set if the data field is valid in response
input [63:0]	data-word-bypass	A store may be bypassed to a read response in the same cycle.
input [63:0]	store-data	Returns the data stored in store request

2.6 Arty FPGA Evaluation Kit

Field Programmable Gate Arrays (FPGAs) are semiconductor devices that are based around a matrix of configurable logic blocks (CLBs) connected via programmable interconnects. FPGAs can be reprogrammed to desired application or functionality requirements after manufacturing. This property make them an ideal starting point for Application Specific Integrated Circuits (ASIC) such as Micro - controller. In this project we use low cost Artix - 7 35T Arty FPGA Evaluation Kit from Digilent. Arty is build around Artix - 35T FPGA from Xillinx. It supports MicoBlaze processor. Artix - 35T has 33,280 logic cells, 5,200 CLB - Slices, 400 Kb of Distributed RAM, 90 DSP Slices, 1800 Kb of Block RAM and 250 User I/O. Arty Evaluation Kit also has 256MB DDR3L, 16MB Quad - SPI Flash along with USB - JTAG Programming capability. The board is supported by Xillinx Vivado Suite for FPGA development in Verilog and VHDL. Xillinx also provide MicroBlaze Software Development Kit (SDK) but it is of no use to us as we are going to implement our own RISC - V processor.

Chapter 3

RISC - V FPGA IMPLEMENTATION

In this chapter we put forward detail of our hardware implementation of RISC - V based 32-bits Micro controller. This implementation is based on Rocket Chip Generator [13] and Freedom Open Source Chip [14]. Rocket Chip Generator is used to generate a 32-bits RISC - V core with Cache Memory, Interrupt Controller, Debug, Peripheral port and System port. Freedom is then used to add peripheral devices on RISC - V SoC generated by Rocket Chip. Both Rocket Chip and Freedom source code are written in Constructing Hardware in a Scala Embedded Language (Chisel). Chisel is an advanced hardware design language that is highly parametrizable and suitable for hardware generators. It is open source and developed at University of California, Berkeley. We start our discussion with Rocket Chip SoC in Section 3.1. We give small introduction of peripheral added by Freedom source code in Section 3.2. The implementation utilizes IP's from Xilinx specific to Arty for clock and reset signal generation. These IP's are discussed in Section 3.3. Finally we give complete detail of design chip in Section 3.4.

3.1 Rocket Chip SoC

RISC - V core is generated by Rocket Chip Generator. It is a 32-bits core based on RV32-I base with Multiplication and Division (M), Atomic (A) and Compression (C) standard extensions. This implementation supports Machine mode. Apart from Integer level computation ALU is capable of performing 8-bits per cycle hardware multiplication and 1-bit per cycle hardware division. Core has a 16KB of 1 - way, 256 - sets Data cache (Dcache) and 64KB of 1 - way, 64 - sets of Instruction cache (Icache) memory. Both caches are supported by 4 entries Translation look-aside buffer (TLB) to further reduce access time to main memory. Core supports 16 direct local Interrupt. It also supports 255 global interrupts through Platform Level Interrupt Controller (PLIC). Local interrupt has higher priority than global interrupt. System has two TileLink platform buses; System Bus and Peripheral Bus. System Bus is used to access main memory to refill caches. Main memory starting address is 0x80000000 and the maximum size supported is 0x10000000. Peripheral bus supports access to peripheral devices with base address and size of 0x20000000. Core has 5 in-order pipeline stages including Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM) and Register Write Back (WB). Core includes Physical Memory Protection Unit which helps to isolate harts and synchronize memory access. It also has two external hardware breakpoints used by debugger. Figure 3.1 shows block diagram of RISC - V Rocket chip level generated SoC.

3.2 Freedom Peripherals

Rocket Chip generates basic structure of SoC with core and interface for Peripheral devices. Actual peripheral on it are added by Freedom source code. Freedom source

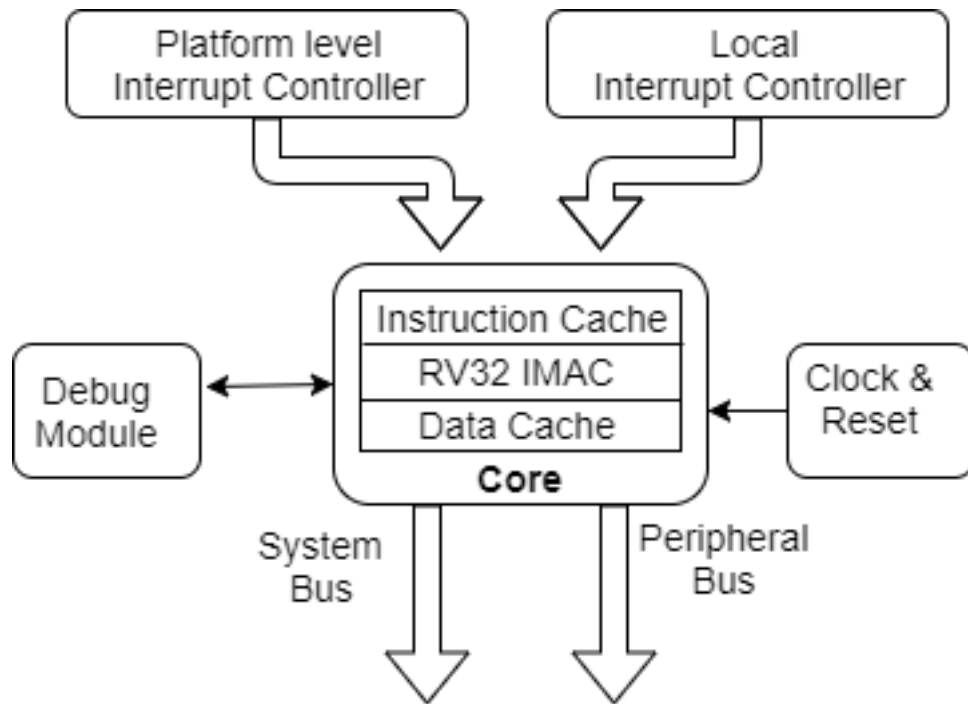


FIGURE 3.1: Rocket Chip SoC Block Diagram

code provide actual hardware implementation code for peripheral such as:

- ♦ General Purpose Input/Output Controller.
- ♦ Serial Peripheral Interface Controller.
- ♦ Universal Asynchronous Receiver/Transmitter.
- ♦ Pulse Width Modulation Controller.
- ♦ I²C interface Controller.
- ♦ Always On Block.
- ♦ Mask Rom.
- ♦ JTAG Debug Interface.

General Purpose Input/Output (GPIO) Controller are used to control the direction of 32 general purpose pins in SoC. System has 32 pins that can be set as input or output, pull high or low using GPIO controller. Interrupt on these pins as well as drive value all are managed by internal memory map GPIO controller.

Serial Peripheral Interface (SPI) Controller provides SPI interface and implementation. This SPI supports master - only operation over the single - lane, dual - lane, and quad - lane protocols. This controller also provide SPI FLASH interface used to flash the program memory with new program.

Universal Asynchronous Receiver/Transmitter (UART) provides 8 - bits data with no parity bit pair UART interface that can supports speed up to 115200 Kbps.

Pulse Width Modulation (PWM) Controller can be used to generate different PWM signals. It controls one 16-bits and one 8-bits counter. Each counter can generates 4 PWM signals.

I²C interface Controller is responsible for the implementation of I²C protocol.

Always On (AON) block includes a Real - Time Counter (RTC), a Watchdog timer (WDT), Backup registers and Power Management Unit (PMU). RTC maintains time for the system and generate interrupts for timed wakeup from sleep - mode or timer interrupts during normal operation. WDT is used to provide a watchdog reset function or a periodic timer interrupt. Backup registers can be used to store critical data during sleep mode. PMU controls system power supplies and reset signals when transitioning into and out of sleep mode. The PMU also monitors AON signals for wakeup conditions.

Mask ROM is a small non volatile memory fixed at design time. On reset Instruction Cache loads command from Mask Rom. Mask ROM contain an instruction that cause jump to One Time Programmable memory (OTP). OTP is actual memory that can be flash using SPI Flash interface.

JTAG Debug interface provides extensive platform - level debug facilities including hardware breakpoints, watch - points, and single - step execution accessed. This debug interface is supported by a full set of open - source debug tools including Openocd and GDB. All essential components in the system can be controlled and monitored over the debug port.

Figure 3.2 shows block diagram of Hardware module implemented by Freedom code.

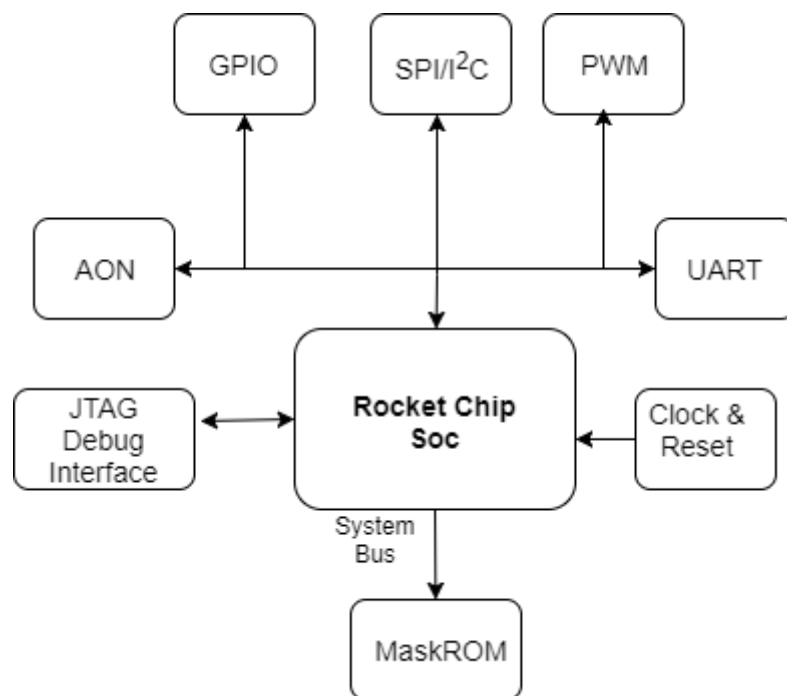


FIGURE 3.2: Freedom Peripheral Block Diagram

3.3 Xilinx IPs for Arty

As shown in Figure 3.1 and Figure 3.2 both block require Clock and Reset. This clock and Reset is implemented using Mixed - Mode Clock Manager (MMCM) and Processor System Reset (ip - reset - sys) IP Module. MMCM generates three clock signals 8 MHz, 32 MHz and 65 Mhz. 8 MHz clock is used by ip - reset - sys for synchronization. 32/65 MHz is used as clock by the main SoC modules and core. Processor Sytem Reset IP provides reset signal depending upon different input reset source such as external power reset, reset by debugger and reset by MMCM.

3.4 Complete System

Figure 3.3 shows the complete block diagram of whole system on ARTY with Hardware Accelerator that we will discuss in Chapter 5. Arty has number of LED's that are being used for Debug purpose. Arty on Chip Debugger is connected to USB port so J-D port is used to map the debug signals.

Table 3.1 shows Pins Direction and Connections on ARTY board. All the inout pin can be used as GPIO input and output but some of them have dedicated purpose as per the peripheral mapping. Table 3.2 shows the address mapping of different peripheral modules. Table 3.3 shows interrupt mapping of Peripheral devices.

TABLE 3.1: Pin Mapping Arty

Direction	Board Pin	SoC Pin
input	CLK100MHZ	100 MHz external clock input
input	clk rst	Active low reset
inout	led0	Shows status of clock reset

TABLE 3.1: Pin Mapping Arty

Direction	Board Pin	SoC Pin
inout	led1	Shows status of JTAG Debugger Pin 6
inout	led2	Shows status of Button 3
inout	led3	GPIO pin 14 output
inout	ledr0	GPIO pin 1 PWM0(1)
inout	ledg0	GPIO pin 2 PWM0(2)
inout	ledb0	GPIO pin 3 PWM0(3)
inout	ledr0	GPIO pin 19 output
inout	ledg0	GPIO pin 21 output
inout	ledb0	GPIO pin 22 output
inout	ledr2	GPIO pin 11 output
inout	ledg2	GPIO pin 12 output
inout	ledb2	GPIO pin 13 output
inout	sw3	Select UART Transmitter source internal clock or data from UART TX
inout	btn0,	GPIO pin 15 input connected to Button 0
inout	btn1,	GPIO pin 30 input connected to Button 1
inout	btn2	GPIO pin 31 input connected to Button 2
inout	btn3	reset
inout	qspi-cs	QSPI pin CS (output)
inout	qspi-sck	QSPI pin SCK (output)
inout	qspi-dq-0	QSPI pin DQ 0 (output)
inout	qspi-dq-1	QSPI pin DQ 1 (output)
inout	qspi-dq-2	QSPI pin DQ 2 (output)
inout	qspi-dq-3	QSPI pin DQ 3 (output)
inout	uart-rxd-out	GPIO pin 17 UART Rx

TABLE 3.1: Pin Mapping Arty

Direction	Board Pin	SoC Pin
inout	uart-txd-in	GPIO pin 16 UART Tx
inout	ja0	GPIO pin 25 input/output
inout	ja1	GPIO pin 24 input/output
inout	jd0	Jtag TDO output
inout	jd2	Jtag TCK input
inout	jd4	Jtag TDI output with pullup
inout	jd5	Jtag TMS output with pullup
inout	jd6	Jtag reset
inout	ck-io-0	GPIO pin 16 output
inout	ck-io-2	GPIO pin 18 output
inout	ck-io-3	GPIO pin 19 output
inout	ck-io-4	GPIO pin 20 output
inout	ck-io-5	GPIO pin 21 output
inout	ck-io-6	GPIO pin 22 output
inout	ck-io-7	GPIO pin 23 output
inout	ck-io-8	GPIO pin 0 PWM0(0)
inout	ck-io-9	GPIO pin 1 PWM0(1)
inout	ck-io-10	GPIO pin 2 SPI CS(0) / PWM0(2)
inout	ck-io-11	GPIO pin 3 SPI MOSI / PWM0(3)
inout	ck-io-12	GPIO pin 4 SPI MISO
inout	ck-io-13	GPIO pin 5 SPI SCK
inout	ck-io-15	GPIO pin 9 SPI CS(2)
inout	ck-io-16	GPIO pin 10 SPI CS(3)
inout	ck-io-17	GPIO pin 11 output
inout	ck-io-18	GPIO pin 12 I ² C SDA

TABLE 3.1: Pin Mapping Arty

Direction	Board Pin	SoC Pin
inout	ck-io-19	GPIO pin 13 I ² C SCL

TABLE 3.2: Peripheral Memory Addresses

Address Range	Type	Peripheral
0 - 1000	RWX	Debug controller
10000 - 12000	R X	ROM
2000000 - 2010000	RW	Local interrupt controller
c000000 - 10000000	RW	Platform level interrupt controller
10000000 - 10001000	RW	AON
10012000 - 10013000	RW	GPIO
10013000 - 10014000	RW	Serial
10014000 - 10015000	RW	Flash SPI
10015000 - 10016000	RW	PWM0
10016000 - 10017000	RW	I ² C
10024000 - 10025000	RW	SPI
10025000 - 10026000	RW	PWM1
80000000 - 80004000	RWX	Memory

TABLE 3.3: Interrupt Map

Interrupt	Peripheral
1 - 2	AON
3	UART
4	SPI
5	Flash SPI
6 - 37	GPIO
38 - 41	PWM0
42 - 45	PWM1
46	I ² C

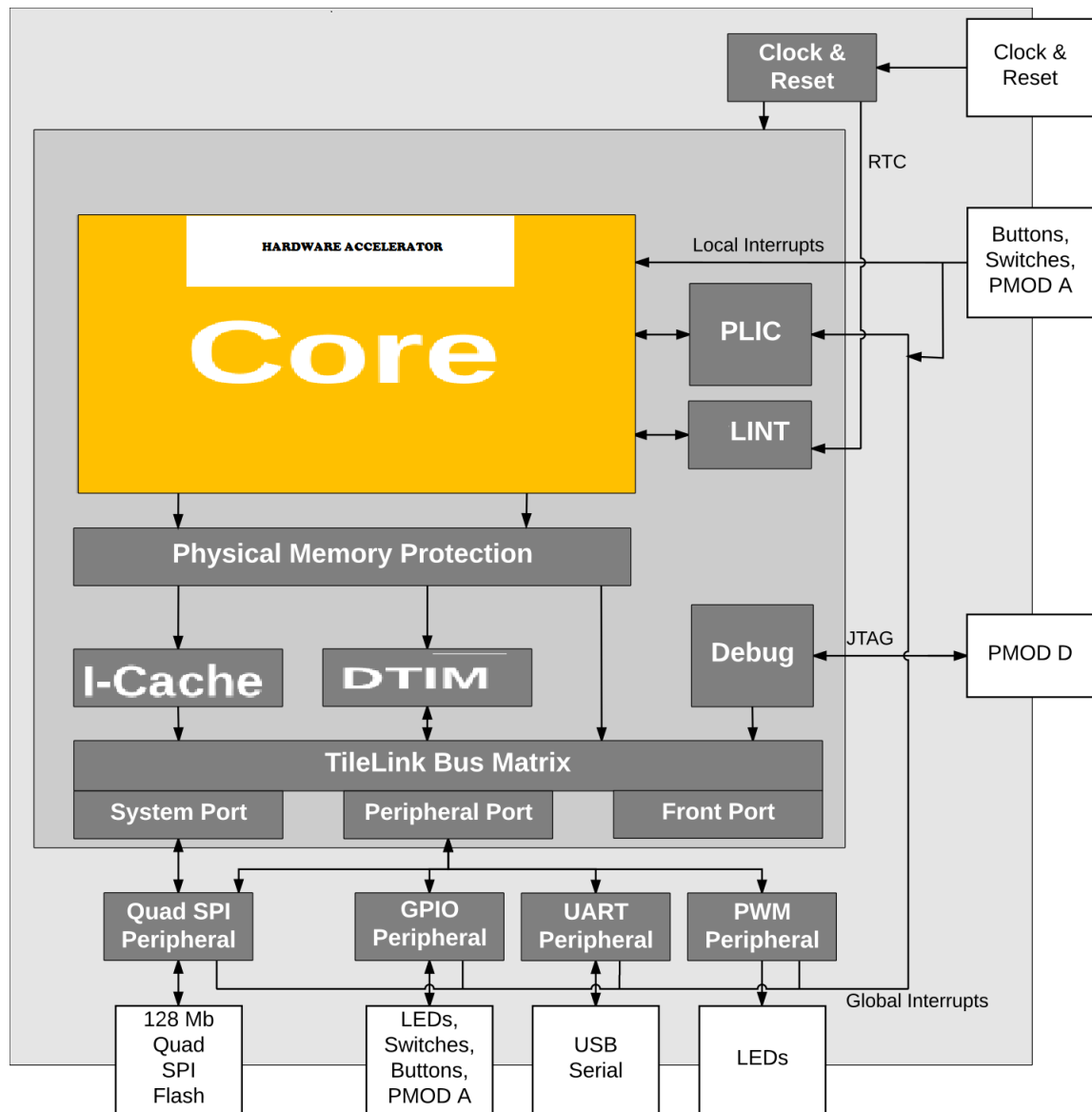


FIGURE 3.3: System Block Diagram

Chapter 4

SOFTWARE IMPLEMENTATION PROFILING

In this chapter we present results of software implementation of ECDSA running on RISC - V based processor. We start our discussion with the execution time results of each steps in ECDSA in [Section 4.1](#) followed by hardware resource utilization in [Section 4.2](#).

4.1 Software Execution Time Results

As discussed in [Section 2.2](#) ECDSA implementation consist of three main sections. In this section we will first analyse each section software implementation to find out sections of codes that take time to execute. [Table 4.1](#) shows execution time taking by each section of code at the core running at 32 MHZ.

ECDSA software implementation makes use of number of routines to implement 256 bits ALU operations using 32 bits ALU. These operations include Addition of two 256

TABLE 4.1: ECDSA Time Profiling

Code Section	Execution Time
Key Pair Generation	3.3734 sec
Signature Generation	3.5498 sec
Signature Verification	4.0974 sec

bits Integers (ADD256), Subtraction of two 256 bits Integers (SUB256), Right logical shift of a 256 - bits integer (RLS256), Modular Addition of two 256 bits integers (MADD256), Modular Subtraction of two 256 bits integers (MSUB256), Modular logical left shift of a 256 bits integer (MLLS256), Modular Multiplication of two 256 bits integers (MMUL256), Modular Inverse of a 256 bits integer (MINV256), Modular Division of two 256 bits integers and Modular Square of a 256 bits number (MSQ). Table 4.2 shows execution time and total number of time each of these routines is called or used by the software.

TABLE 4.2: Time Profiling of Subroutines

Operations	Execution Time	Called
ADD256	4.8333 μ s	95459
SUB256	4.4167 μ s	58525
RLS256	3.6667 μ s	5763
MADD256	10.8333 μ s	82150
MSUB256	4.7083 μ s	23042
MLLS256	9.5833 μ s	22882
MMUL256	289.3 μ s	7245
MINV256	6395.9 μ s	4
MDIV256	5461.7 μ s	3
MSQ	264.6 μ s	4196

In Table 4.2 subroutines ADD256, SUB256, RLS265 and MLLS256 can be considered as a basic subroutines. All other subroutines can be obtained by the combination of these 4 routines. So we can improve the performance of whole system if we could improve the performance of these 4 operations. So we will target Addition of two 256 bits integers, Subtraction of two 256 bits integers, Logical left shift of 256 bit

integer and Logical right shift of 256 bit integer in our Custom Accelerator discussed in Chapter 5.

4.2 Resource Utilization

Our RISC - V implementation consumes about 63.89% of Slice Logic on Arty without the Accelerator. Table 4.3 gives total consumption of Logical Look Up Table (LUT), Look Up Table as Distributed RAM (LUTRAMs), Shift Register Look Up Table (SRLs), Flip Flops (FFs), Block RAM (RAMB36 and RAM18) and DSP slice (DSP48).

TABLE 4.3: Resource Utilization

Resource	Utilization
Logic LUTs	12286
LUTRAMs	296
SRLs	32
FFs	7185
RAMB36	5
RAMB18	1
DSP48 Blocks	2

Chapter 5

HARDWARE ACCELERATOR

Hardware Accelerator are used to accelerate the performance. Software implementation of ECDSA and results presented in Chapter 4 show that to accelerate the ECDSA with 256 bits curve we need an ALU that can operate on 256 bits inputs. As per result presented in Table 4.2 we design 256 bits Adder in Section 5.1, 256 bits Subtractor in Section 5.2 and 256 bits Logical left and right Barrel shifter in Section 5.3. We show the complete implemented accelerator in Section 5.4 followed by new custom instructions details in Section 5.5.

5.1 Adder 256 Bits

Adder consist of one 1 - bit Full Adder at input and 255 single bit parallel adders. Parallel adder compute single bit addition with carry equal to zero and carry equals to 1. We then use 1 - bit multiplexer for each sum and carry. This way of addition is similar to ripple carry adder. But in ripple carry adder next stage sum is calculated depending upon the carry from the previous stage but here the next stage addition is already performed by parallel adder and carry is only used to select the correct result. The logic

equation for Single Bit Full Adder Sum and Carry output is shown in Equations 5.1 - 5.2.

$$Sum = A \oplus B \oplus Cin \quad (5.1)$$

$$Cout = Cin.(A \oplus B) + A.B \quad (5.2)$$

For Parallel Adder we need to perform addition with carry equal to zero and carry equal to one. This enable us to simplify the logic equations for parallel addition as shown in Equations 5.3 - 5.6.

$$Sum_0 = A \oplus B \quad (5.3)$$

$$Cout_0 = A \& B \quad (5.4)$$

$$Sum_1 = !Sum_0 \quad (5.5)$$

$$Cout_1 = A + B \quad (5.6)$$

where Sum_0 , Sum_1 , $Cout_0$ and $Cout_1$ are sum with carry in 0, sum with carry in 1, carry out with carry in 0 and carry out with carry in 1 respectively. 1 - bit multiplexer is a simple multiplexer that select output (out) from input 1 (in1) and input 2 (in2) depending upon logic value of select (sel) input as depicted by Equation 5.7.

$$out = (sel \& in1) | (!sel \& in2) \quad (5.7)$$

The block diagram of full 256 bits adder is shown in Figure 5.1.

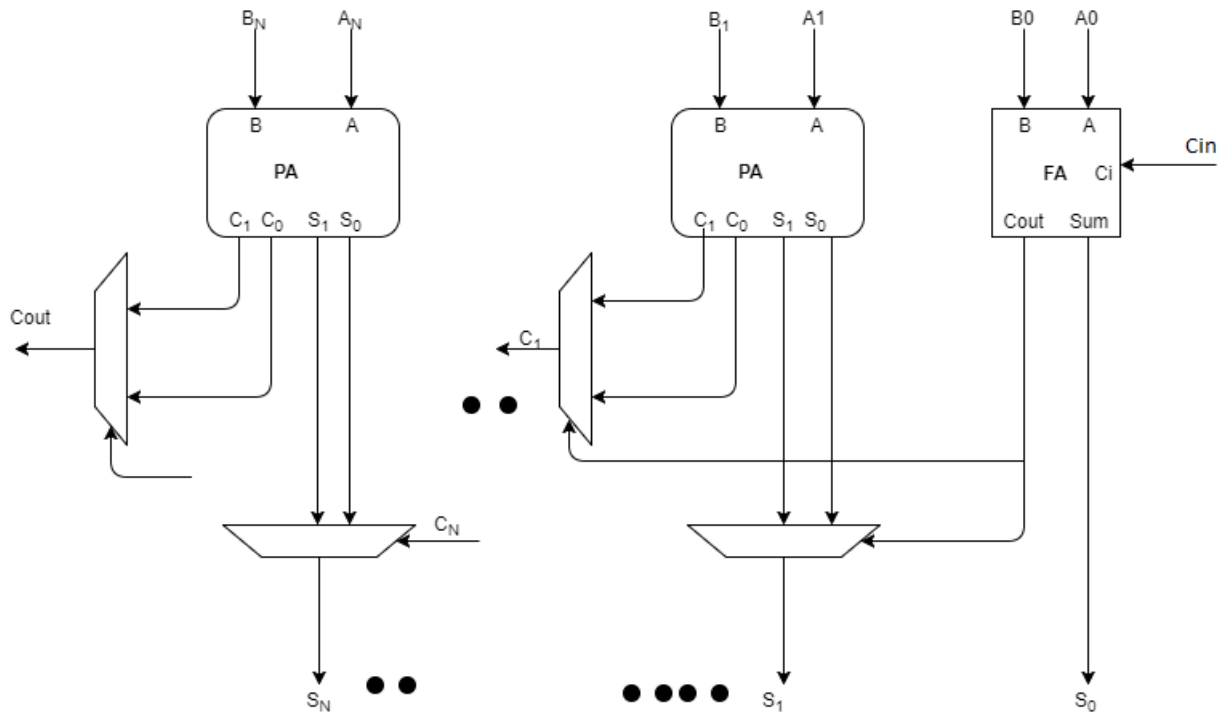


FIGURE 5.1: Adder 256 Block Diagram

5.2 Subtractor 256 Bits

Subtractor implemented here perform subtraction in 2's Complement form. This means that we can use Adder as subtractor if we invert all bits of value to be subtracted and keep input carry as one. The output carry obtain through addition needs to be inverted to. We can change the Adder 256 bits circuit to Adder and Subtractor by adding Multiplexer at input B and Cin, take a signal add as input and add a multiplexer at carry out. Circuit work such that when add input is high it act as Adder and input B value are passed but when add input is low all input B bits are inverted as well as Carry out. With add signal at low Carry in is also tied to 1 as shown in Figure 5.2.

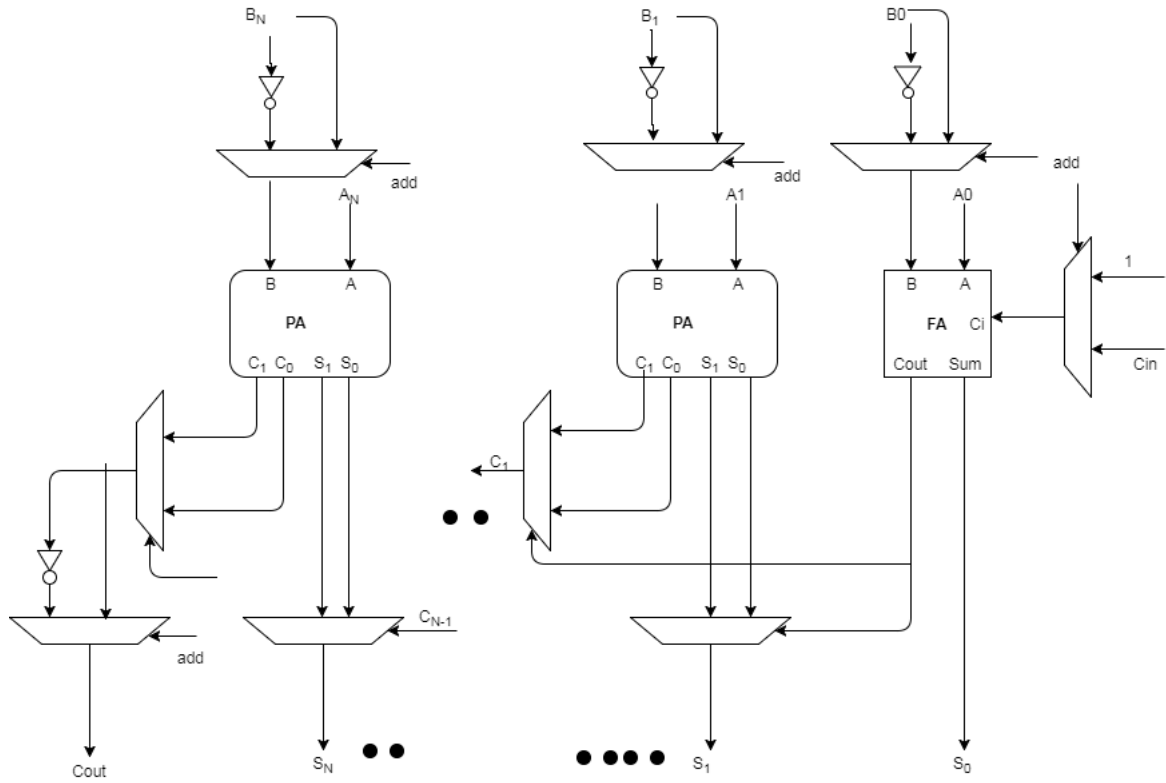


FIGURE 5.2: Adder & Subtractor 256 Block Diagram

5.3 Barrel Shifter 256 Bits

In order to perform logical left and right shift operation we implement a Barrel shifter that can shift input shifts amount of time depending upon the selected input and direction. The shifter is implemented using 16 Columns of 256 rows of 1 - bit multiplexer given by Eq 5.7. 8 Columns are used to actually implement the shift operation while the other 8 columns help to decide the direction of shift. In logical left shift LSB's are filled with zero and in logical right shift MSB's are filled with zero as shown in Figure 5.3. Single bit input signal I_r is used to select the direction of shift while the amount of shift is provided by 8 bits input signal S .

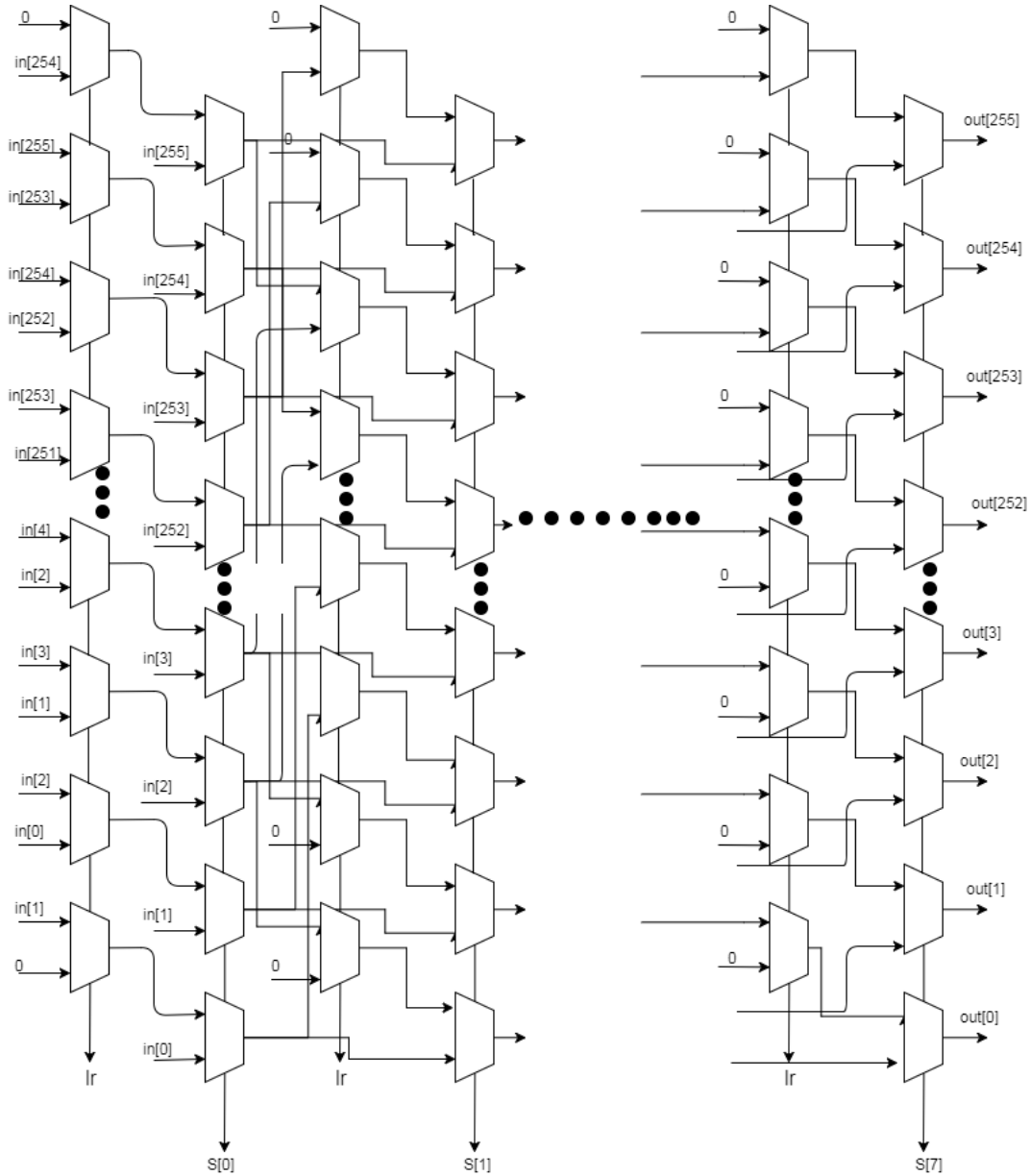


FIGURE 5.3: Barrel Shifter 256 Block Diagram

5.4 Complete Accelerator

We can combine Adder and Subtractor circuit of Figure 5.2 with Barrel shifter of Figure 5.3 to obtain complete accelerator. Integration of these two circuit require multiplexer at the output as shown in Figure 5.4 where the thick black lines show 256 bits bus. A

and **B** are 256 bits input signals to operate on. **Out** is 256 bits output result. Shamt [7:0] is an eight bits signal that provide amount of logical shift to be performed. Single bit signals Add, Sub, sr and sl are used to select Addition, Subtraction, Right shift and Left shift respectively.

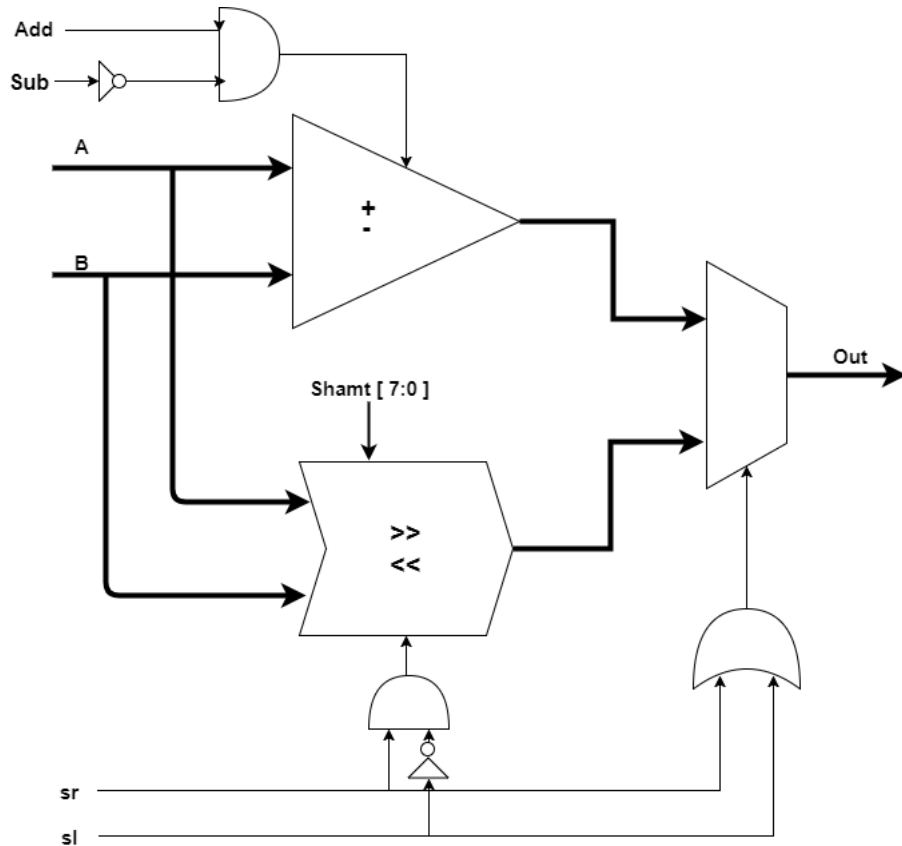


FIGURE 5.4: Accelerator Block Diagram

5.5 Accelerator New Instructions

The accelerator supports 7 new instructions. The data path from main core to accelerator and from main memory to accelerator is of 32 bits width. This means that even though the accelerator can perform operation on 256 bits, data is passed through and from accelerator in 32 bits form. To ease arithmetic operations we make 4 256

bits registers R1 - R4 in accelerator. Operand for Addition and Subtraction should be written in register R1 and R2. Result is written into register R3. Operand for Logical shift operations should be written in register R1. Register R4 is used to store temporary values. Below we discuss all the seven instructions involve in details.

5.5.1 doWrite

doWrite is used by the main core to write 32 - bits data to the accelerator register R1 - R4. Since R1 - R4 are 256 bits register therefore we need 8 doWrite to write all 256 bits to any one of Register R1 - R4. Instruction format for doWrite is shown below:

doWrite (Dummy register, Register address, Data)

For example to write 32 bits integer data "P" to register R1 first 32 bits the instructions is:

doWrite (y, R1, P)

where y is a dummy 32 bits register needed to make the instructions consistent. Similarly to write data to R2 5th 32 bits data position we have:

doWrite (y, R2+4, P)

We also have a variant of doWrite by the name of doWriteC which is used to write Carry in for addition operation. For example to set carry in to 1 for addition operation we have:

doWriteC(y,0x1)

5.5.2 doRead

doRead is used by the main core to read 32 - bits data from the accelerator register R1 - R4. Since R1 - R4 are 256 bits register therefore we need 8 doRead to read all 256 bits from any one of Register R1 - R4. Instruction format for doRead is shown below:

doRead (Core Register, Register address)

For example to read 32 bits integer data from accelerator register R1 to main core register P we have:

doRead (P, R1)

after the instruction P now has same value as R1 first 32 bits. Similarly to read data from R3 8th 32 bits data position we have:

doRead (P, R3+7)

We also have a variant of doRead by the name of doReadC which is used to read Carry out in addition and subtraction operations. For example to read carry out we have:

doReadC(carry,Cout)

where carry is the main core register.

5.5.3 doCopy

doCopy is used to internally copy one 256 bits data register to another 256 bits register. Since main core can only read 32 bits data at a time therefore it is not feasible to always read out answer in the main core particularly when we know that we have to use the result obtain in next operations. Instruction set format for doCopy is as follows:

doCopy (dummy register, Register, Register)

For example to copy result from R3 to be used in for next logical operation we have:

doCopy (y, R1, R3)

where y is a dummy 32 bits register as mention before.

5.5.4 doAdd

doAdd is used to do addition operations on operand written in registers R1 and R2 using doWrite. The addition is performed with carry in set using doWriteC. The result is stored in register R3 and can be read using doRead. Carry out in addition operation is read using doReadC as mention before. Instruction format for doAdd is as follows:

doAdd (Dummy register)

For example to do addition we have:

doAdd (y)**5.5.5 doSub**

doSub is used to do subtraction operations on operand written in registers R1 and R2 using doWrite. The result is stored in register R3 and can be read using doRead. doSub subtracts register R2 from R1. Results is in 2's complement form and sign bit can be read using doReadC. Instruction format for doSub is as follows:

doSub (Dummy register)

For example to do subtraction we have:

doSub (y)

5.5.6 doSL

doSL is used to do logical left shift operations on operand written in registers R1 using doWrite. The result is stored in register R3 and can be read using doRead. Amount of shift is provided by 8 bits register value. Instruction format for doSL is as follows:

doSL (Dummy register, 8 bits integer values)

For example to do logical left shift by 0xFF (256) amount of time we have:

doSL (y, 0xFF)

5.5.7 doSR

doSR is used to do logical right shift operations on operand written in registers R1 using doWrite. The result is stored in register R3 and can be read using doRead. Amount of shift is provided by 8 bits register value. Instruction format for doSR is as follows:

doSR (Dummy register, 8 bits integer values)

For example to do logical right shift by 0xF (16) amount of time we have:

doSR (y, 0xF)

Table [5.1](#) summarize all the new instructions describe above.

TABLE 5.1: Accelerator Instruction Summary

Instruction	Example	Description
doWrite	doWrite (y, R1, P)	Write Accelerator register R1 first 32 bits with value in P.
doWriteC	doWrite (y,0x0)	Write Carry in equal to zero
doRead	doRead (P, R1)	Read Accelerator register R1 first 32 bits in P
doReadC	doReadC (P, Cout)	Read Carry out or sign in P
doCopy	doCopy(R2,R4)	Copy Accelerator register R4 into R2
doAdd	doAdd(y)	Add Accelerator register R1 into R2 with carry in.
doSub	doSub(y)	Subtracr Accelerator register R2 from R1
doSL	doSL(y,P)	Logical shift Accelerator register R1 P amount of time to left.
doSR	doSR(y,P)	Logical shift Accelerator register R1 P amount of time to right.

Chapter 6

HARDWARE ACCELERATOR PROFILING

In this chapter we present ECDSA software implementation results in the presence of Accelerator. We start our discussion with execution time taking by each new instructions in Section 6.1 followed by results of time taking by each routines used to implement ECDSA. We present resource utilization results of Accelerator on Arty in Section 6.2.

6.1 Accelerator Hardware Execution Time Results

In this section we present the actual hardware timing results for the design accelerator. We start our discussion with the seven new instructions as mention in section 5.5. Table 6.1 shows execution time taking by each of this instruction.

doWrite and doRead deals with 32 bits data. To move 256 bits of data we need 8 instructions of doWrite or doRead. We named these new functions as WriteAcc and

TABLE 6.1: Accelerator Instruction Timing Profile

Instruction	Execution Time
doWrite	0.5 μ s
doRead	0.5 μ s
doCopy	0.5 μ s
doAdd	0.5 μ s
doSub	0.5 μ s
doSL	0.625 μ s
doSR	0.625 μ s

ReadAcc. WriteAcc writes 256 bits of data from main core to accelerator register R1 - R4. ReadAcc reads 256 bits of data from accelerator register R1 - R4 into the main core. Table 6.2 shows execution time taking by WriteAcc and ReadAcc.

TABLE 6.2: WriteAcc & ReadAcc Execution Time

Instruction	Execution Time
WriteAcc	2.9583 ms
ReadAcc	4.4583 ms

We can use WriteAcc and ReadAcc along with doAdd, doSub, doSR and doSL to generate the operation equivalent to operations shown in Table 4.2. Table 6.3 shows execution time taking by these equivalent operations. Execution time from Table 4.2 are copied here again for reader feasibility.

Execution time shown by Table 6.3 appears to be similar or in some cases not better than the one shown in Table 4.2. This is because the timing of these routines are quoted with WriteAcc and ReadAcc. In order to benefit from the presence of Accelerator we can reduce unnecessary Read and Write to the accelerator. For example suppose that we have to perform modular addition of 256 bits integer a, b and c. If we do it using MADD256 routines. Than we first add a and b then ReadAcc the result in d. After that we again use MADD256 to add d and c to obtain the final result using

TABLE 6.3: Time Profiling of Subroutines with Accelerator

Operations	Execution Time with Accelerator	Execution Time without Accelerator
ADD256	9.333 μ s	4.8333 μ s
SUB256	9.125 μ s	4.4167 μ s
RLS256	6.792 μ s	3.6667 μ s
MADD256	11.083 μ s	10.8333 μ s
MSUB256	9.417 μ s	4.7083 μ s
MLLS256	8.542 μ s	9.5833 μ s
MMUL256	708.9 μ s	289.3 μ s
MINV256	10883.3 μ s	6395.9 μ s
MDIV256	7237.5 μ s	5461.7 μ s
MSQ	986.4 μ s	264.6 μ s

ReadAcc. This way we have an unnecessary ReadAcc in first step and WriteAcc in second step. We can remove this unnecessary ReadAcc and WriteAcc by combining two MADD256 steps. We name the new function as AddToAdd which basically first Writes two operands to the Accelerator registers R1 and R2, perform additions then copy result from R3 to R1 and write third operand in R2 and do Addition again and finally read the answer using ReadAcc. We can write similar routine for Subtraction followed by subtraction (SubToSub), Addition followed by multiplication (AddToMul) and Subtraction followed by multiplication. Table 6.4 shows the execution time taking by these routines.

TABLE 6.4: Combine Routines Time Profiling

Routine	Execution Time
AddToAdd	15.25 μ s
SubToSub	12.625 μ s
AddToMul	912.5 μ s
SubToMul	718.9 μ s

With the introduction of these new routines the time taking by each section of ECDSA is given in Table 6.5. Results from Table 4.1 are copied here again for comparison.

TABLE 6.5: ECDSA Time Profiling with Accelerator

Code Section	Execution Time with Accelerator	Execution Time without Accelerator
Key Pair Generation	2.9752 sec	3.3734 s
Signature Generation	3.0842 sec	3.5498 s
Signature Verification	4.1073 sec	4.0974 s

Although the time difference in Table 6.5 and Table 4.1 is not that much but the software routines for ECDSA with Accelerator are easy to write as compare to software routine written on 16/32/64 bit ALU to obtain 256 bits ALU operation.

6.2 Accelerator Resource Utilization

Our Accelerator implementation consumes about 12.14% of Slice Logic on Arty without the RISC - V actual core. Table 6.6 gives total consumption of Logical Look Up Table (LUT), Look Up Table as Distributed RAM (LUTRAMs), Shift Register Look Up Table (SRLs), Flip Flops (FFs), Block RAM (RAMB36 and RAM18) and DSP slice (DSP48).

TABLE 6.6: Resource Utilization Accelerator

Resource	Utilization
Logic LUTs	7247
LUTRAMs	106
SRLs	0
FFs	600
RAMB36	0
RAMB18	0
DSP48 Blocks	0

Chapter 7

CONCLUSION AND RECOMMENDATION FOR FUTURE WORK

In this chapter we summarised and conclude our work in Section 7.1. We end this thesis with direction and recommendation for future work in Section 7.2.

7.1 Conclusion

In this thesis we showed that RISC - V ISA can be used to design Hardware Accelerator for complex application such as ECDSA running on micro controller. As an example we built a prototype RISC - V micro controller on Xilinx Arty FPGA Evaluation Kit that support instruction favourable to the software implementation of ECDSA.

We started our discussion in Chapter 1 with the introduction of ECDSA and RISC - V. In Chapter 1 we formulated a problem that ECDSA is suitable for embedded system

but there is still a need of micro controller for embedded system that support ECDSA implementation. In Chapter 2 we summarized and review literature on Elliptic Curve Cryptography, Elliptic Curve Digital Signature Algorithm (ECDSA) and RISC - V. We presented summary of RISC - V 32 bits Integer Instruction base (RV32I). RV32I along with standard Atomic, Compression, Multiplication and Division Extensions form the processor core of our micro controller. We ended Chapter 2 with discussion about the RISC - V custom accelerator instruction and interface which makes it easy to design Hardware Accelerator for RISC - V based core without much changes in actual core and software design.

We presented complete detail of our implemented RISC - V based 32 bits micro controller without Accelerator on Arty in Chapter 3. Like all other general micro controller this controller also support peripheral such as GPIO, Serial, UART, PWM, I²C and JTAG. We showed results of execution time taken by ECDSA software implementation on RISC - V based processor in Chapter 4. We summarized all the main Arithmetic and logical operation require by ECDSA software routines in Table 4.2. From Chapter 4 we concluded that for ECDSA implementation we require four basic 256 bits ALU operations namely Addition, Subtraction, Logical left shift and right shift.

In Chapter 5 we gave complete theoretical detail of each component of Hardware Accelerator including Adder, Subtractor and Barrel Shifter. To reduce the resource utilization we combine circuit of Adder and Subtractor using Multiplexer. Similarly we combine Left Shifter and Right Shifter for the same reason. Finally output from both circuit is integrated using multiplexer as shown in Figure 5.4. We presented complete detail of newly implemented instruction doWrite, doRead, doCopy, doAdd, doSub, doSL and doSR in section 5.5. We tested our implemented hardware accelerator in Chapter 6. Chapter 6 results showed that Hardware Accelerator utilizes only 12.14 % of resource on Arty and gives performance a bit better than without Accelerator. Inclusion of Hardware Accelerator greatly simplify the complex software routines without

any degrade in performance.

7.2 Recommendation for Future Work

This thesis work is our first attempt on RISC - V based system. Time and efforts were put into establishing working RISC - V software ecosystem. There is number of ways the system can be improve in future. Two of them are discussed here.

1. We can improve the system by modifying the Hardware Accelerator. The current Hardware Accelerator is design to minimize the hardware resources. We can improve the Hardware Accelerator by implementing modular arithmetic directly in it or by implementing time and resource consuming multiplication and division in it. Modification of Hardware Accelerator means that we will have to change the prototyping Arty FPGA evaluation Kit with the kit that has more hardware slice such as Kintex FPGA Evaluation Kit. But this will require changes in RISC - V software ecosystem to make it compatible with Kintex.
2. Performance of the Accelerator is degraded mainly due to data path. Implemented Accelerator can operate on 256 bits at one time but can read and write only 32 bits data to the main core which each takes $0.5 \mu s$ limiting the overall system through put. There are 2 ways data path can be improve. First rather than designing 32 bits main RISC - V core we can design 64/128 bits main RISC - V core. This will definitely improve overall system performance but will also result in more hardware resource consumption. Another way to improve the performance is to change the RoCC interface data path between main core and Hardware Accelerator. This is easy to do in hardware but require tuning and effort in software ecosystem.

Bibliography

- [1] M.Ann and M.David, *The Role of Cryptography in Security for Electronic Commerce*. The ITB Journal, Vol. 2, Iss. 1, Article 3, PP. 21 - 50, May 2001.
- [2] Z.J.Shi and H.Yan, *Software Implementations of Elliptic Curve Cryptography*, International Journal of Network Security, Vol.7, No.1, PP.141–150, July 2008.
- [3] Ghanmy Nabil et al., *Hardware implementation of Elliptic Curve Digital Signature Algorithm (ECDSA) on Koblitz Curves*, in IET 8th International Symposium on Communication Systems, Networks and Digital Signal Processing, 2012, PP. 1 - 6.
- [4] B.Panjwani and D.C.Mehta, *Hardware-Software Co-design of Elliptic Curve Digital Signature Algorithm over Binary Fields*, International Conference on Advances in Computing, Communications and Informatics (ICACCI), 2015, PP. 1101 - 1106.
- [5] Ghanmy Nabil et al., *Enhancement security level and hardware implementation of ECDSA*, IEEE Symposium on Computers and Communications (ISCC), 2013, PP. 000423 - 000429.

-
- [6] A.Abidi, B.Bouallegue and F.Kahri, *Implementation of Elliptic Curve Digital Signature Algorithm (ECDSA)*, Global Summit on Computer & Information Technology (GSCIT) , 2014,PP 1 - 6.
 - [7] A.Sghaier, M.Zeghid and M.Machhout, *FAST HARDWARE IMPLEMENTATION OF ECDSA SIGNATURE SCHEME*, International Symposium on Signal, Image, Video and Communications (ISIVC), 2016, Pp 342 - 348.
 - [8] Anissa Sghaier et al., *Design and Implementation of Low Area/Power Elliptic Curve Digital Signature Hardware Core*,MDPI - Electronics Journal, Vol. 6, Iss .2, Pp 46 - 69,2017.
 - [9] Benjamin Glas et al., *Prime Field ECDSA Signature Processing for Reconfigurable Embedded Systems*,International Journal of Reconfigurable Computing, Volume 2011, Article ID 836460, 12 pages, 2011.
 - [10] Noura Ben Hadjy Youssef et all. *A Low-Resource 32-bit Datapath ECDSA Design for Embedded Applications*, International Carnahan Conference on Security Technology (ICCST), 2014, PP 1 - 6.
 - [11] K.Asanović and D.A.Patterson. *Instruction Sets Should Be Free: The Case For RISC-V*, Electrical Engineering and Computer Sciences, University of California at Berkeley, Report No. UCB/EECS-2014-146, Aug 2014.
 - [12] RISC - V Foundation. (2017). RISC - V [Online]. Available: <https://riscv.org/> [Accessed: 15 Dec.2017].
 - [13] freechipsproject/rocket-chip (2017). Rocket Chip Generator [Online] Available: <https://github.com/freechipsproject/rocket-chip> [Accessed: 15 Dec.2017].
 - [14] sifive/freedom (2017). Source files for SiFive's Freedom platforms [Online] Available: <https://github.com/sifive/freedom> [Accessed: 15 Dec.2017].

-
- [15] Anoop MS, *Elliptic Curve Cryptography:An implementation Guide*, InfoSecWriters, Article, 22 - 06 - 2015.
 - [16] Elaine Barker, *Digital Signature Standard (DSS)*, National Institute of Standards and Technology (NIST), FIPS PUB 186-4, July 2013.
 - [17] A.Waterman¹ and K.Asanović, Eds, *The RISC-V Instruction Set Manual, Volume I: User-Level ISA*, RISC - V Foundation, SiFive Inc., CS Division, EECS Department, University of California, Berkeley, Document Version 2.2. May 2017.
 - [18] A.Waterman¹ and K.Asanović, Eds, *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture*, RISC - V Foundation, SiFive Inc., CS Division, EECS Department, University of California, Berkeley, Document Version 1.10. May 2017.
 - [19] Anuj Rao. *An Introduction to the Rocket Custom Coprocessor Interface* Taylor's Bespoke Silicon Group & UCSD, The RoCC Doc V2.